UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

DEPARTAMENTO: ELECTRICA, ELECTRONICA Y TELECOMUNICACIONES
CARRERA: TELECOMUNICACIONES *NRC:* 36466
ASGINATURA: FUNDAMENTOS DE PROGRAMACION *FECHA:* 25/10/2026
NOMBRE: PAUCAR MENDOZA MARIA FERNANDA
TEMA: EJERCICIOS DE PROGRAMACION Y ALGORITMOS

Problema 1.1. Media de dos valores

Desarrolla un programa que ingrese dos números por teclado e imprima su media.

Análisis del problema

- Entradas: Dos números de tipo numérico (pueden tener 2decimales).
- Proceso: Sumar ambos números y dividir el resultado entre 2.
- Salida: Un número que representa la media calculada.

Tabla de objetos

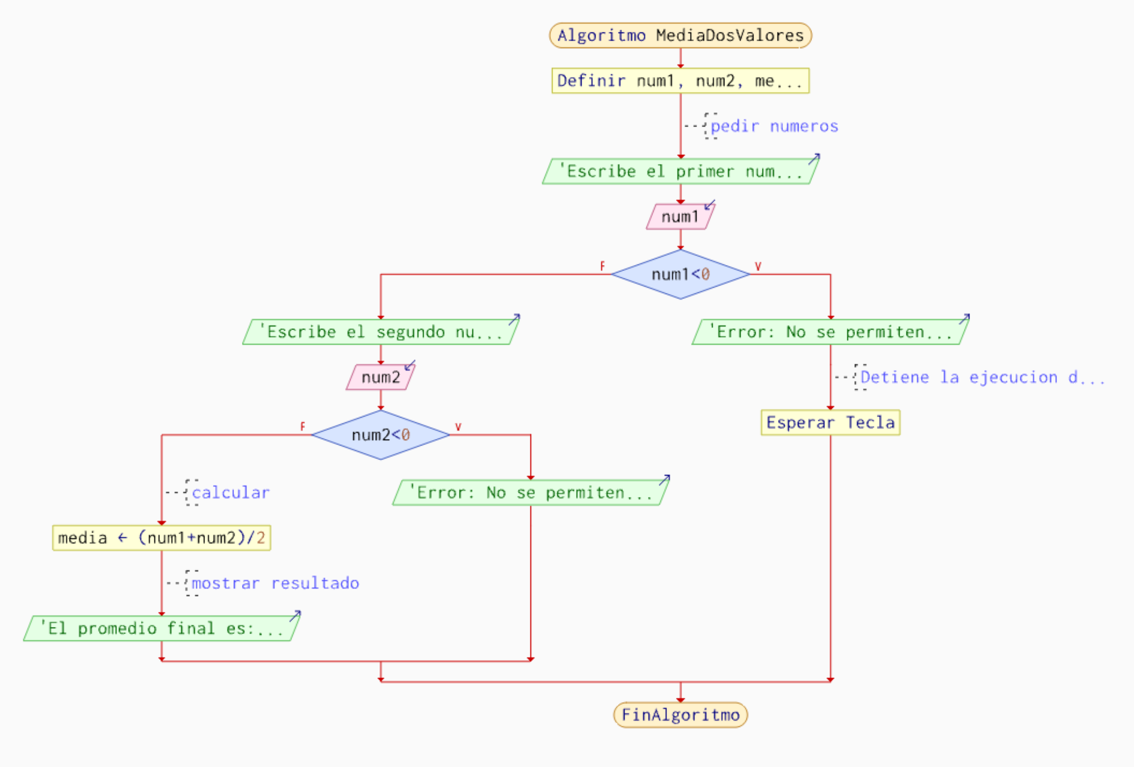
OBJETO	NOMBRE	VALOR	TIPO
Variable	num1	Primer numero ingresado	Float
Variable	num2	Segundo numero ingresado	Float
Variable	media	(num1+num2)/2.0	float

Seudocódigo

```
Algoritmo MediaDosValores
    Definir num1, num2, media Como Real
    Escribir "Ingrese  el primer numero: "
    Leer num1
    Si num1 < 0 Entonces
        Escribir "Error: No se permiten numeros negativos."
    Sino
        Escribir "Ingrese  el segundo numero: "
        Leer num2
        Si num2 < 0 Entonces
            Escribir "Error: No se permiten numeros negativos."
        Sino
            media <- (num1 + num2) / 2
            Escribir "El promedio final es: ", media
        FinSi
    FinSi
```

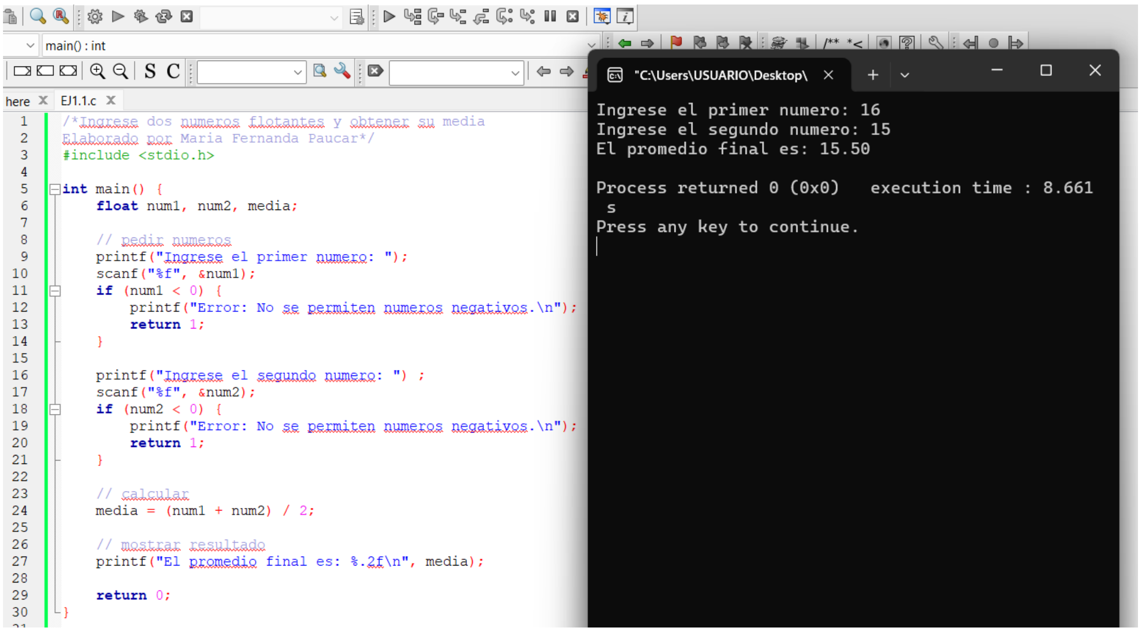
```
FinAlgoritmo
```

Diagrama de flujo


Algoritmo MediaDosValores
Definir num1, num2, me...
pedir numeros
'Escribe el primer num...
num1
num1<0
F
V
'Escribe el segundo nu...
num2
'Error: No se permiten...
Detiene la ejecucion d...
num2<0
F
V
Esperar Tecla
calcular
'Error: No se permiten...
media ← (num1+num2)/2
mostrar resultado
'El promedio final es:...
FinAlgoritmo

Prueba de escritorio

PASO	num1	num2	Media	Salida
1				Ingrese el primer numero
2	16			
3				Ingrese el segundo numero
4	16	15	15.50	El promedio final es: 15.50
5	4			Ingrese el primer numero
6		-58		Ingrese el segundo numero
7	4	-58	_	No se permiten números negativos



```c
/*Ingrese dos numeros flotantes y obtener su media
Elaborado por Maria Fernanda Paucar*/
#include <stdio.h>

int main() {
    float num1, num2, media;

    // pedir numeros
    printf("Ingrese el primer numero: ");
    scanf("%f", &num1);
    if (num1 < 0) {
        printf("Error: No se permiten numeros negativos.\n");
        return 1;
    }

    printf("Ingrese el segundo numero: ") ;
    scanf("%f", &num2);
    if (num2 < 0) {
        printf("Error: No se permiten numeros negativos.\n");
        return 1;
    }

    // calcular
    media = (num1 + num2) / 2;

    // mostrar resultado
    printf("El promedio final es: %.2f\n", media);

    return 0;
}
```

```
Ingrese el primer numero: 16
Ingrese el segundo numero: 15
El promedio final es: 15.50

Process returned 0 (0x0)   execution time : 8.661
 s
Press any key to continue.
```

Problema 1.2. Valor absoluto de x al cubo

Desarrolla un programa que lea un número por teclado y escriba por pantalla |x|³.

Análisis del problema

- Entrada: un numero real “x”
- Restricción: si el numero es menor a cero o negativo , se muestra un mensaje de error y el programa termina
- Proceso: obtener el valor absoluto del numero en este caso, al verificar que es positivo, el valor absoluto es el mismo numero y elevarlo a la potencia 3

Tabla de objetos

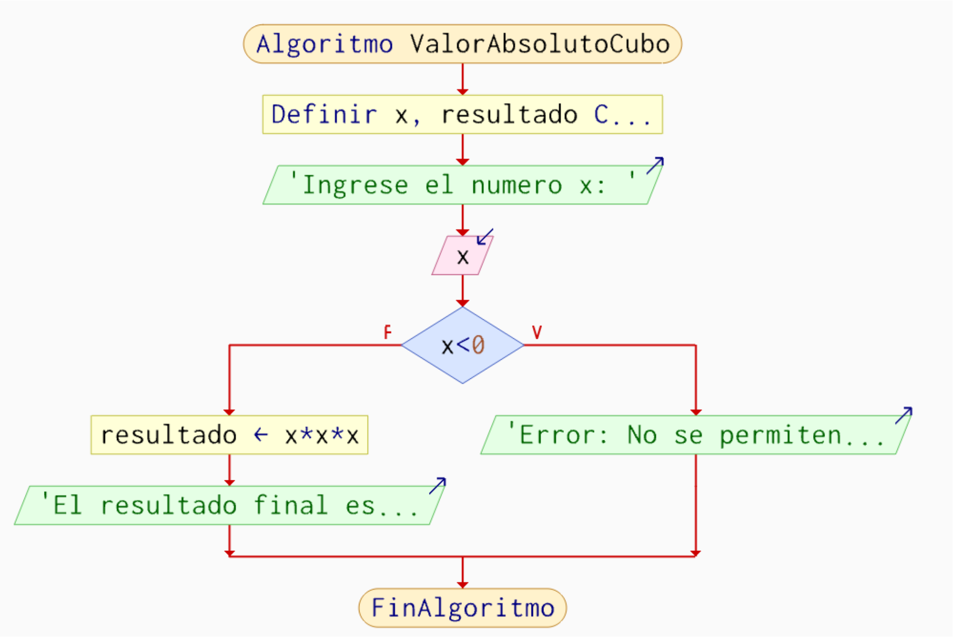
OBJETO	NOMBRE	VALOR	TIPO
Variable	x	Numero ingresado por el usuario	Float
Variable	resultado	Almacena el calculo de x^3	Float

Seudocódigo

```
Algoritmo ValorAbsolutoCubo

    Definir x, resultado Como Real

    Escribir "Ingrese el numero x: "

    Leer x

    Si x < 0 Entonces

        Escribir "Error: No se permiten numeros negativos."

    Sino
```

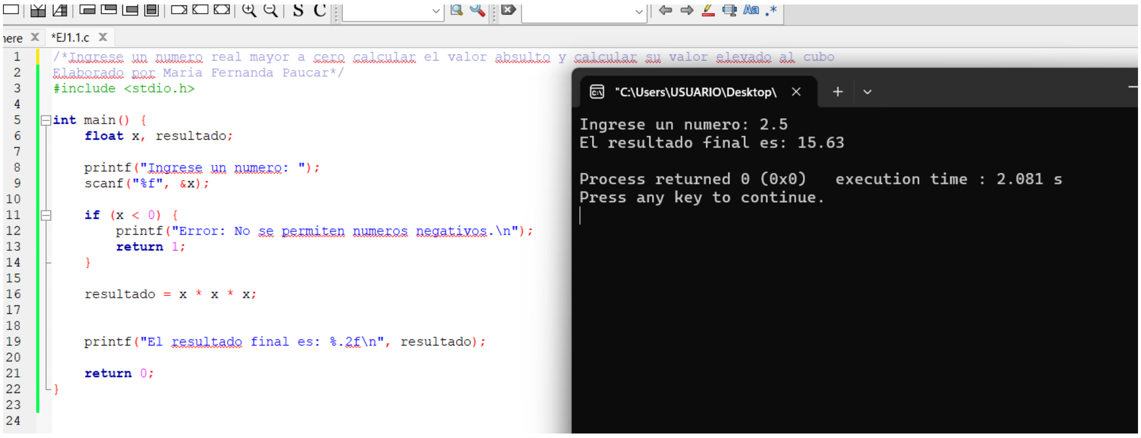
```
            resultado <- x * x * x
            Escribir "El resultado final es: ", resultado
        FinSi
FinAlgoritmo
```

Diagrama de flujo


Algoritmo ValorAbsolutoCubo
Definir x, resultado C...
'Ingrese el numero x: '
x
F
x<0
V
resultado ← x*x*x
'Error: No se permiten...
'El resultado final es...
FinAlgoritmo

Prueba de escritorio

PASO	X	RESULTADO	SALIDA
1			Ingrese un numero x
2	3		Por teclado ingresa 3
3	3	27	Se realiza la validación que 3<0 es falsa continua
4	3	27	El resultado final es: 27
5			Ingrese un numero x
6	-8		Por teclado ingresa -8
7	-8	---------	Validación -8<5 es verdadera
8	-8	---------	Error: no se permite números negativos
9	2.5		Por teclado ingresa 2.5
10	2.5	15.63	El resultado final es: 15.63



```c
/*Ingrese un numero real mayor a cero calcular el valor absulto y calcular su valor elevado al cubo
Elaborado por Maria Fernanda Paucar*/
#include <stdio.h>

int main() {
    float x, resultado;

    printf("Ingrese un numero: ");
    scanf("%f", &x);

    if (x < 0) {
        printf("Error: No se permiten numeros negativos.\n");
        return 1;
    }

    resultado = x * x * x;


    printf("El resultado final es: %.2f\n", resultado);

    return 0;
}
```

```
Ingrese un numero: 2.5
El resultado final es: 15.63

Process returned 0 (0x0)   execution time : 2.081 s
Press any key to continue.
```

Problema 1.3. Divisible

Desarrolla un programa que lea dos números enteros por teclado y determine si el primero es divisible por el segundo. Se mostrará por pantalla el resultado.

Pista: La solución utiliza el operador módulo (%), que devuelve el resto de la división entera entre números enteros. Si x es divisible entre y, el resto debe ser 0.

Análisis del problema

- Dos números enteros num1 y num2
- Restricciones: cualquiera de los dos números es menor a cero se muestra un mensaje de error y el programa finaliza
- Procesos: verificar su el resto de la división entera 2 mod y es igual a 0 usando el operador modulo %
- Salida: mensaje indica si es divisible o no.

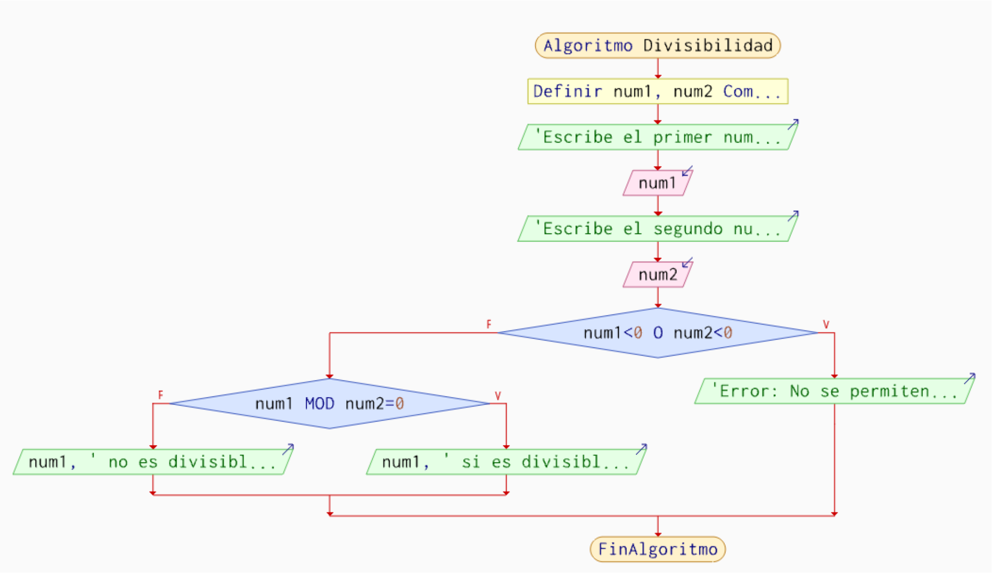
Tabla de objetos

OBJETO	NOMBRE	VALOR	TIPO
Variable	Num1	Primer numero entero	Int
Variable	Num2	Segundo numero entero	Int

Seudocódigo

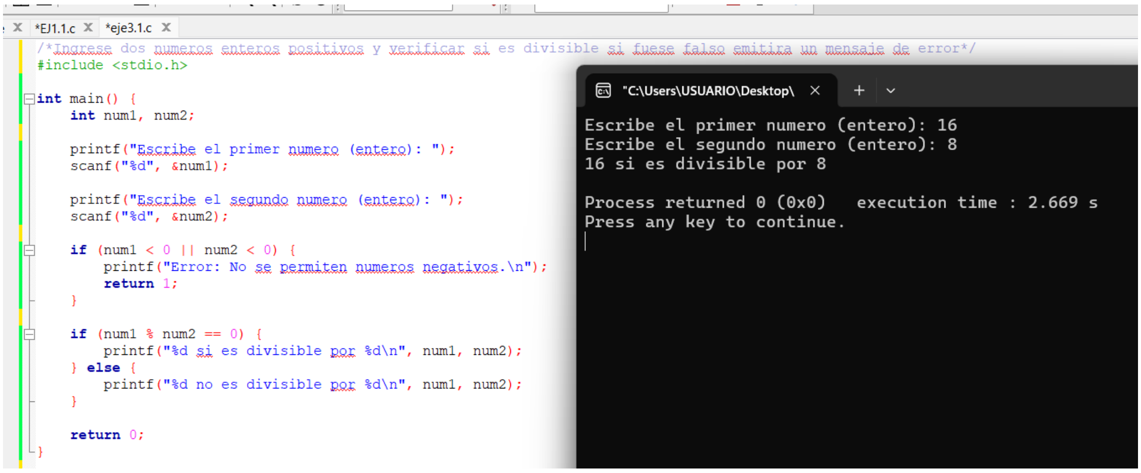
```
Algoritmo Divisibilidad
    Definir num1,num2 Como Entero

    Escribir "Escribe el primer numero (entero): "
    Leer num1
    Escribir "Escribe el segundo numero (entero): "
    Leer num2

    Si num1 < 0 O num2 < 0 Entonces
        Escribir "Error: No se permiten numeros negativos."
    Sino
        Si num1 Mod num2 = 0 Entonces
            Escribir num1, " si es divisible por ", num2
        Sino
            Escribir num1, " no es divisible por ", num2
        FinSi
    FinSi
FinAlgoritmo
```

Diagrama de flujo


Algoritmo Divisibilidad
Definir num1, num2 Com...
'Escribe el primer num...
num1
'Escribe el segundo nu...
num2
num1<0 O num2<0
F
V
'Error: No se permiten...
num1 MOD num2=0
F
V
num1, ' no es divisibl...
num1, ' si es divisibl...
FinAlgoritmo

Prueba de escritorio

PASO	Num1	Num2	Salida
1	16		Ingreso el primer numero
2	16	8	Ingreso el segundo numero
3	16	8	Verdadero (16<0 o 8<0 es falsa)
4	16	8	16 si es divisible por 8
5	5		Ingreso el primer numero
6	5	4	Ingreso el segundo numero
7	5	4	Falso
8	5	4	5 no es divisible para 4
9	-2		Ingreso el primer numero
10	-2	6	Ingreso el segundo numero
11	-2	6	Erro: no se permiten numero negativos



```c
/*Ingrese dos numeros enteros positivos y verificar si es divisible si fuese falso emitira un mensaje de error*/
#include <stdio.h>

int main() {
    int num1, num2;

    printf("Escribe el primer numero (entero): ");
    scanf("%d", &num1);

    printf("Escribe el segundo numero (entero): ");
    scanf("%d", &num2);

    if (num1 < 0 || num2 < 0) {
        printf("Error: No se permiten numeros negativos.\n");
        return 1;
    }

    if (num1 % num2 == 0) {
        printf("%d si es divisible por %d\n", num1, num2);
    } else {
        printf("%d no es divisible por %d\n", num1, num2);
    }

    return 0;
}
```

```
Escribe el primer numero (entero): 16
Escribe el segundo numero (entero): 8
16 si es divisible por 8

Process returned 0 (0x0)   execution time : 2.669 s
Press any key to continue.
```

Problema 1.4. Intervalo

Desarrolla un programa que lea un número por teclado y determine si pertenece al intervalo [0, 10], indicando el resultado por pantalla.

Pista: El número introducido debe cumplir x ≥ 0 y x ≤ 10 (ambas condiciones a la vez) para pertenecer al intervalo. Para establecer estas dos condiciones se utiliza un operador lógico: el operador Y en el diagrama de flujo y && en C. Evita la tentación de escribir 0 ≤ x ≤ 10 como una sola condición, porque en C no se evalúa como esperas.

Análisis del problema

- Entrada: un numero ingresado por teclado
- Proceso: verificar mediante una condición lógica si el numero es mayor o igual a 0 y menor o igual a 10
- Salida: un mensaje en pantalla que indicque si el numero pertenece o no al intervalo [0,10]

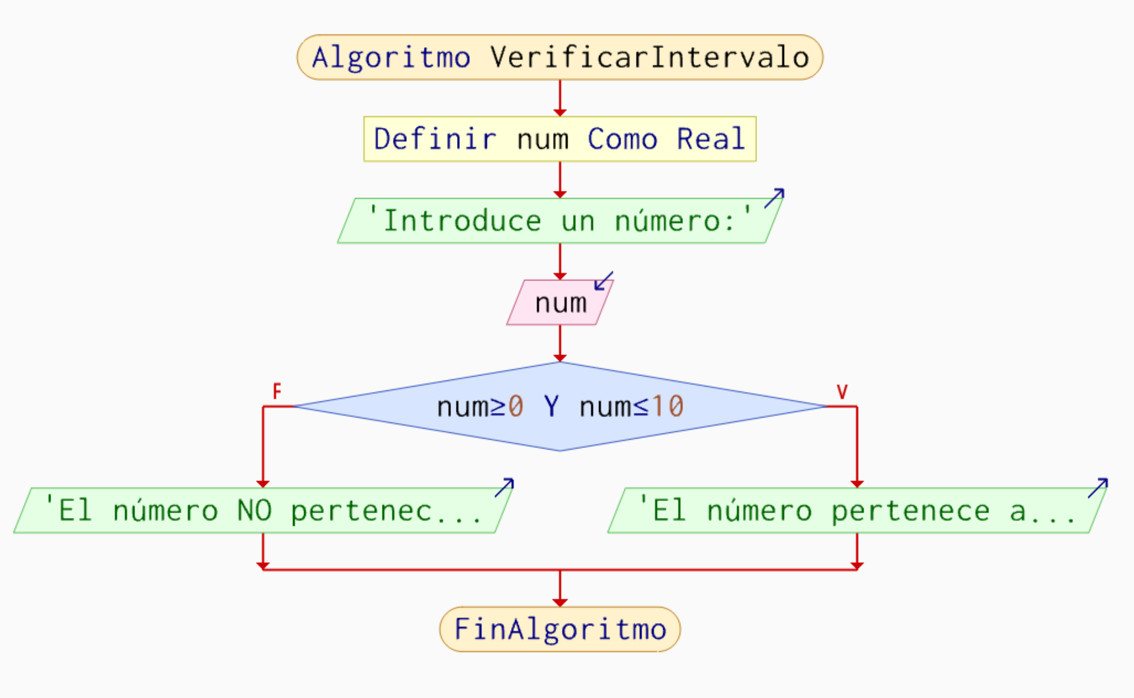
Tabla de objetos

OBJETO	NOMBRE	VALOR	TIPO
Variable	Num	Numero ingresado por teclado	Int
Mensaje de éxito	El numero pertenece al intervalo [0,10]	Texto	Cadena
Mensaje de error	El numero NO pertenece al intervalo [0,10]	Texto	Cadena

Seudocódigo

```
Algoritmo VerificarIntervalo

  Definir num Como Real

  Escribir "Introduce un número:"

  Leer num

  Si num >= 0 Y num <= 10 Entonces

    Escribir "El número pertenece al intervalo"
```

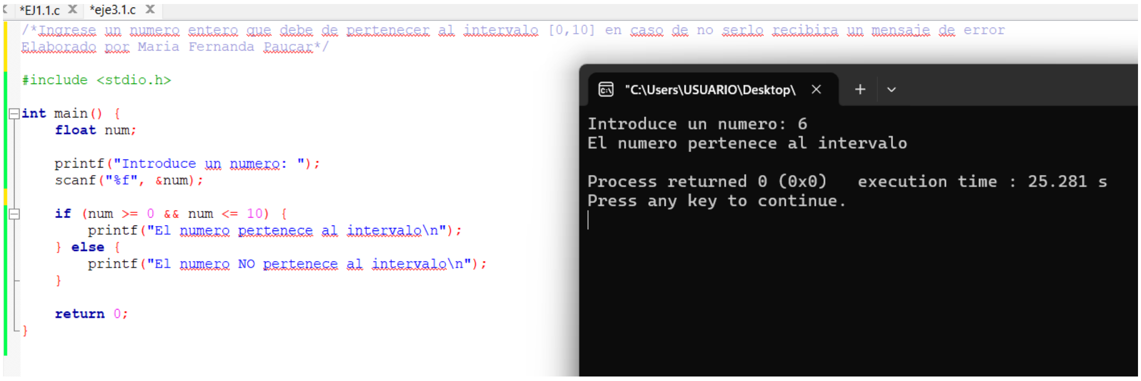
```
Sino
    Escribir "El número NO pertenece al intervalo"
FinSi
```


Algoritmo VerificarIntervalo
Definir num Como Real
'Introduce un número:'
num
num≥0 Y num≤10
F
V
'El número NO pertenec...
'El número pertenece a...
FinAlgoritmo

FinAlgoritmoDiagrama de flujo

Prueba de escritorio

Paso	Valor	Condición (num>= 0 Y mun<=10)	Salida
1	6	6>=0 V Y 6<=10 V	El numero pertenece al intervalo [0,10]
2	-16	16>=0 F Y 16<=10 F	El numero no pertenece al intervalo [0,10]
3	45	45>=0 F Y 45<=10 F	El numero no pertenece al intervalo [0,10]


*EJ1.1.c
*eje3.1.c
"C:\Users\USUARIO\Desktop\

```c
/*Ingrese un numero entero que debe de pertenecer al intervalo [0,10] en caso de no serlo recibira un mensaje de error
Elaborado por Maria Fernanda Paucar*/

#include <stdio.h>

int main() {
    float num;

    printf("Introduce un numero: ");
    scanf("%f", &num);

    if (num >= 0 && num <= 10) {
        printf("El numero pertenece al intervalo\n");
    } else {
        printf("El numero NO pertenece al intervalo\n");
    }

    return 0;
}
```

```
Introduce un numero: 6
El numero pertenece al intervalo

Process returned 0 (0x0)   execution time : 25.281 s
Press any key to continue.
```

Problema 1.5. Conversión de unidades de tiempo

Desarrolla un programa que lea por teclado un valor entero c, interpretándolo como el número de segundos que dura un evento, y calcule y muestre por pantalla cuántos días, horas, minutos y segundos representa.

Para calcular los días, horas, minutos y segundos se utilizan la división entera y el módulo (resto de una división, operador %). Sea c el número de segundos a convertir. Como un día tiene 24 horas, una hora tiene 60 minutos y un minuto tiene 60 segundos, el cociente entero c / (24 · 60 · 60) es el número de días completos contenidos en c. Asimismo, c % (24 · 60 · 60) es el número de segundos restantes tras descontar esos días. Se procede de igual manera para obtener las horas (1 hora = 60 · 60 segundos) y los minutos (1 minuto = 60 segundos).

Ejemplo: supón que quieres convertir c = 10 000 segundos. Como 10 000 / (24 · 60 · 60) = 0, hay 0 días, porque 10 000 segundos es menos que un día. Análogamente, 10 000 / (60 · 60) = 2 horas. Si a 10 000 segundos les quitas esas 2 horas, quedan 10 000 % (60 · 60) = 2 800 segundos, que son algo más de 46 minutos, cantidad que se obtiene con 2 800 / 60 = 46 minutos. Finalmente, 2 800 % 60 = 40 segundos. Por lo tanto, 10 000 segundos equivalen a 0 días, 2 horas, 46 minutos y 40 segundos.

Análisis del problema

- Entrada: un valor entero que represente los segundos totales
- Proceso: calcular los días: división entera de los segundos entre los segundos que tiene un día (24 horas).
  obtener el resto de segundos después de calcular los días usando el operador modulo (% o MOD).
  calcular las horas división entera del resto anterior entre los segundos de una hora.
  obtener el nuevo resto de segundos tras calcular las horas.
  calcular los minutos división entera del ultimo resto entre los segundos de un minuto.
  obtener los segundos restantes el residuo final de la división anterior.
- Salida: días, horas, minutos y segundos resultantes.

Tabla de objetos

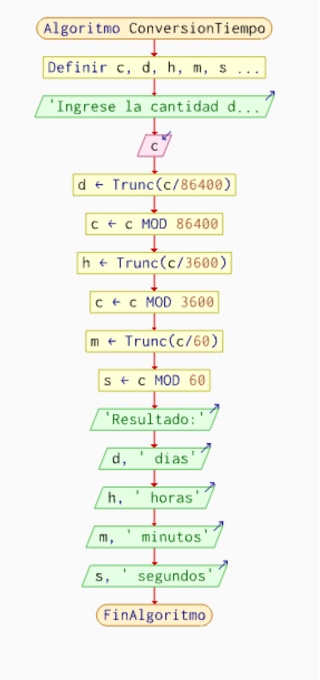
OBJETO	NOMBRE	VALOR	TIPO
Variable	C	Segundos totales introducido por el usuario	Entero
Variable	Días	Cantidad de días completos	Entero
Variable	Horas	Cantidas de horas completas restantes	Entero
Varianle	Minutos	Cantidad de minutos completos restantes	Entero
Variable	Segundos_finales	Segundos sobrantes finales	Entero
Variable	Resto	Segundos sobrantes en los pasos intermedios	Entero

Seudopcodigo

```
Algoritmo ConversionTiempo

Definir c, d, h, m, s Como Entero
```

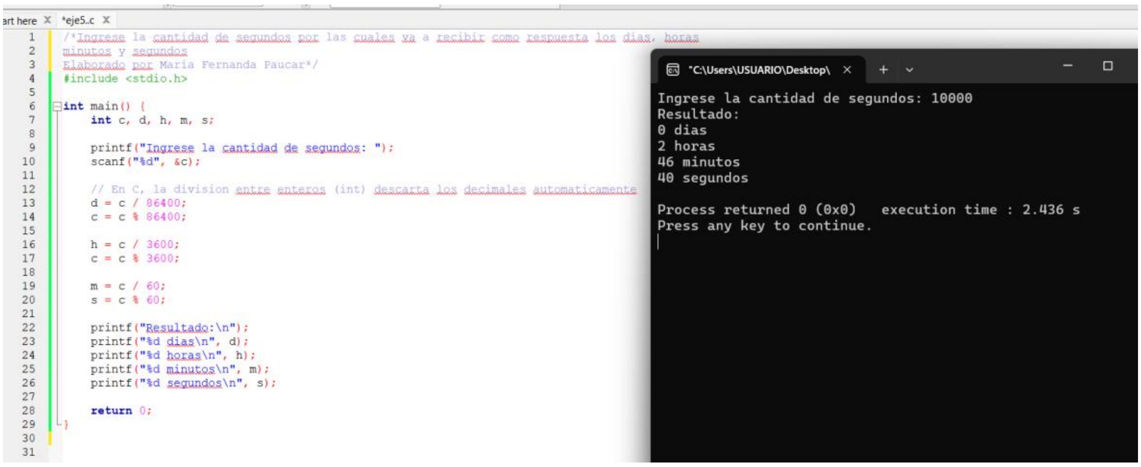
```
Escribir "Ingrese la cantidad de segundos:"
Leer c


d <- Trunc(c / 86400)
c <- c Mod 86400

h <- Trunc(c / 3600)
c <- c Mod 3600

m <- Trunc(c / 60)
s <- c Mod 60

Escribir "Resultado:"
Escribir d, " dias"
Escribir h, " horas"
Escribir m, " minutos"
Escribir s, " segundos"
FinAlgoritmo
```

Diagrama de flujo


Algoritmo ConversionTiempo
Definir c, d, h, m, s ...
'Ingrese la cantidad d...
c
d ← Trunc(c/86400)
c ← c MOD 86400
h ← Trunc(c/3600)
c ← c MOD 3600
m ← Trunc(c/60)
s ← c MOD 60
'Resultado:'
d, ' dias'
h, ' horas'
m, ' minutos'
s, ' segundos'
FinAlgoritmo

Prueba de escritorio

Acción	C	D	H	M	S	Salida
%d,&c	10000	-----	------	-----	-------	Ingrese la cantidad de segundos
d=c /86400	10000	0	---	------	------	100000/86400 da como resultado 0
c=c % 86400	10000	0	----	----	-----	El resto sigue siendo 1000
h=c /3600	10000	0	2	----	------	10000/3600 da como enteros el 2
c=c %3600	2800	0	2	.-----	------	Da como resultado 2800
m=c /60	2800	0	2	46	------	Da como resultado el entero 46
s=c %60	2800	0	2	46	40	La división es 40
Printf (..)	2800	0	2	46	40	Resultado: 0 días 2 horas 46 minutos 40 segundos



```c
/*Ingrese la cantidad de segundos por las cuales va a recibir como respuesta los dias, horas
minutos y segundos
Elaborado por Maria Fernanda Paucar*/
#include <stdio.h>

int main() {
    int c, d, h, m, s;

    printf("Ingrese la cantidad de segundos: ");
    scanf("%d", &c);

    // En C, la division entre enteros (int) descarta los decimales automaticamente
    d = c / 86400;
    c = c % 86400;

    h = c / 3600;
    c = c % 3600;

    m = c / 60;
    s = c % 60;

    printf("Resultado:\n");
    printf("%d dias\n", d);
    printf("%d horas\n", h);
    printf("%d minutos\n", m);
    printf("%d segundos\n", s);

    return 0;
}
```

```
Ingrese la cantidad de segundos: 10000
Resultado:
0 dias
2 horas
46 minutos
40 segundos

Process returned 0 (0x0)   execution time : 2.436 s
Press any key to continue.
```

Problema 1.6. Máximo, mínimo y media de alturas

Desarrolla un programa que lea por teclado un número indeterminado de alturas y deje de leer cuando se introduzca una negativa. El programa debe calcular e imprimir la media, el máximo y el mínimo de las alturas introducidas

Análisis del problema

- Entradas: un numero indeterminado de alturas números decimales reales, el programa se detiene cuando la altura es menor a 0
- Proceso: se lee las alturas dentro del bucle
  validación de parada: si el usuario ingresa un numero menor que 0, el bucle se detiene inmediatamente. Ese numero negativo se descartara por completo y no entra en los cálculos
  para las alturas validas mayores a 0 se suman su valor en un asumulador sum y se cuenta cuantas veces van en un contador cont
  se compara cada altura con el max y el min actuales para mantener el registro de la mas alta y la mas baja

- Salida: el promedio real (media=suma/ cantidad) la altura máxima y la mínima de las estaturas validas.

Tabal de objetos

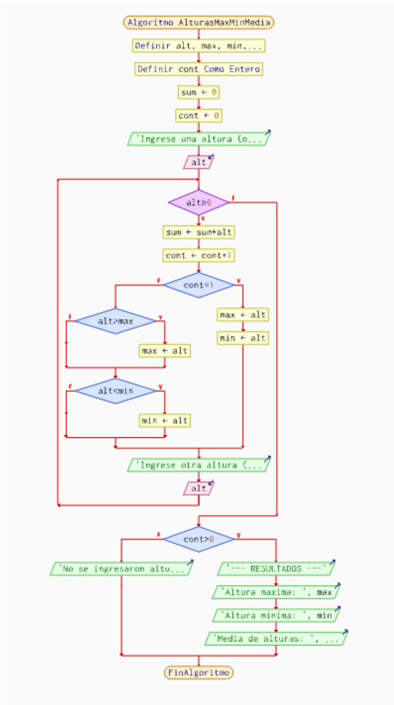
OBJETO	NOMBRE	VALOR	TIPO
Variable	Alt	Altura ingresada por el usuario (si es negativa termina el programa)	Real
Variable	Max	La altura mas grande guardada	Real
Variable	Min	La altura mas pequeña guardada	Real
Variable	Sum	Suma de todas las alturas reales/validas	Real
Variable	Cont	Contador de cuantas alturas validas se ingresaron	Entero

Seudocódigo

```
Algoritmo AlturasMaxMinMedia
    Definir alt, max, min, sum Como Real
    Definir cont Como Entero
    sum <- 0
    cont <- 0
    Escribir "Ingrese una altura (o un numero negativo para salir):"
    Leer alt

    Mientras alt >= 0 Hacer
        sum <- sum + alt
        cont <- cont + 1

        Si cont = 1 Entonces
            max <- alt
            min <- alt
        Sino

            Si alt > max Entonces
                max <- alt
```

```
                    FinSi
                    Si alt < min Entonces
                        min <- alt
                    FinSi
                FinSi

                Escribir "Ingrese otra altura (o un numero negativo para salir):"
                Leer alt
            FinMientras

            Si cont > 0 Entonces
                Escribir "--- RESULTADOS ---"
                Escribir "Altura maxima: ", max
                Escribir "Altura minima: ", min
                Escribir "Media de alturas: ", (sum / cont)
            Sino
                Escribir "No se ingresaron alturas validas."
            FinSi
        FinAlgoritmo
```

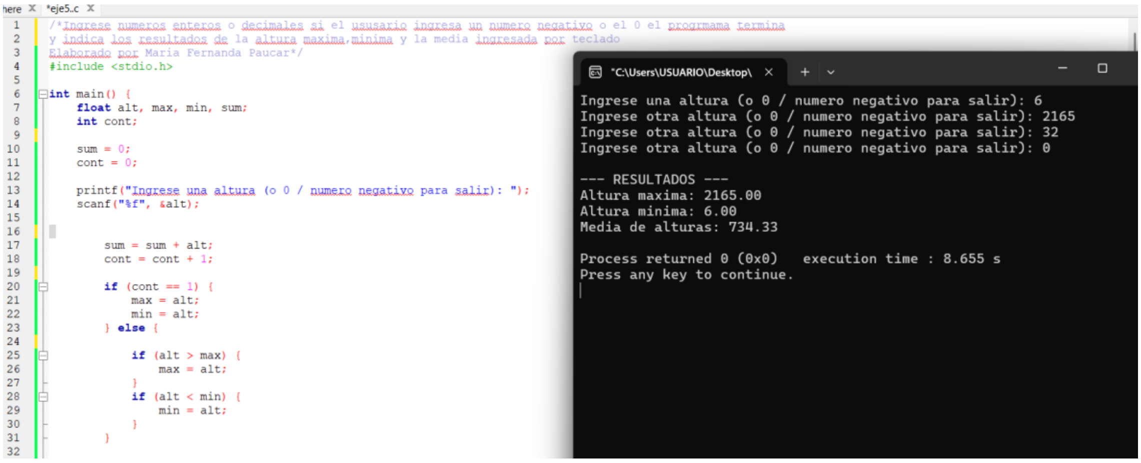
Diagrama de flujo



Algoritmo AlturasMaxMinMedia
Definir alt, max, min,...
Definir cont Como Entero
sum ← 0
cont ← 0
'Ingrese una altura (o...
alt
alt≥0
F
V
sum ← sum+alt
cont ← cont+1
cont=1
F
V
alt>max
F
V
max ← alt
max ← alt
min ← alt
alt<min
F
V
min ← alt
'Ingrese otra altura (...
alt
cont>0
F
V
'No se ingresaron altu...
'--- RESULTADOS ---'
'Altura maxima: ', max
'Altura minima: ', min
'Media de alturas: ', ...
FinAlgoritmo

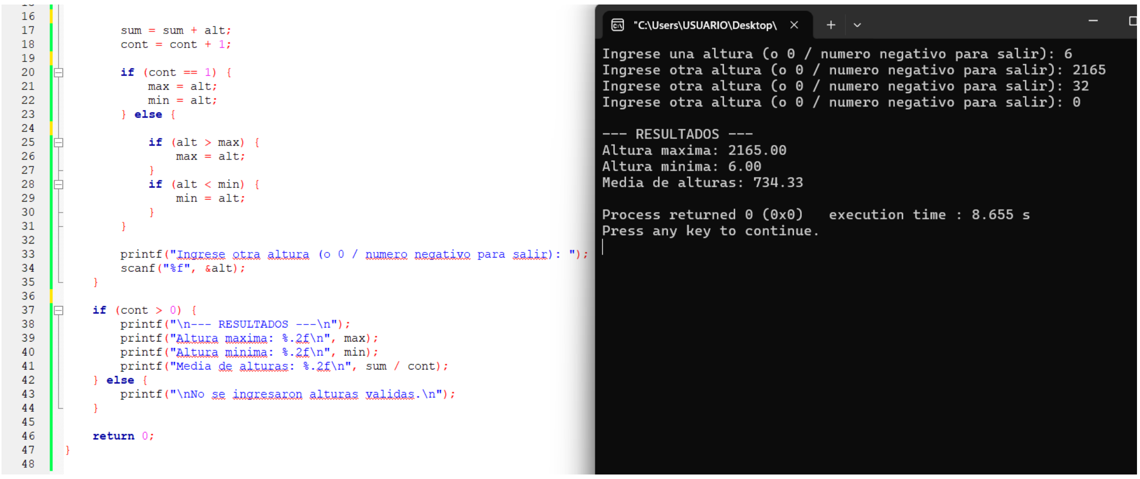
Prueba de escritorio

PASO	Alt	Max	Min	Sum	Cont	Salida
Inicio	-----	------	-----	0	0	----
Ingresa altura: 6	6	6	6	6	1	----
Segunda altura: 2165	2165	2165	6	2171	2	
Tercera altura: 32	32	2165	6	2203	3	
Cuarta altura:0	0	2165	6	2203	3	Altura max: 2165 min: 6 media: 734.33



```c
/*Ingrese numeros enteros o decimales si el ususario ingresa un numero negativo o el 0 el progrmama termina
y indica los resultados de la altura maxima,minima y la media ingresada por teclado
Elaborado por Maria Fernanda Paucar*/
#include <stdio.h>

int main() {
    float alt, max, min, sum;
    int cont;

    sum = 0;
    cont = 0;

    printf("Ingrese una altura (o 0 / numero negativo para salir): ");
    scanf("%f", &alt);


        sum = sum + alt;
        cont = cont + 1;

        if (cont == 1) {
            max = alt;
            min = alt;
        } else {

            if (alt > max) {
                max = alt;
            }
            if (alt < min) {
                min = alt;
            }
        }

```

```
Ingrese una altura (o 0 / numero negativo para salir): 6
Ingrese otra altura (o 0 / numero negativo para salir): 2165
Ingrese otra altura (o 0 / numero negativo para salir): 32
Ingrese otra altura (o 0 / numero negativo para salir): 0

--- RESULTADOS ---
Altura maxima: 2165.00
Altura minima: 6.00
Media de alturas: 734.33

Process returned 0 (0x0)   execution time : 8.655 s
Press any key to continue.
```



```c

        sum = sum + alt;
        cont = cont + 1;

        if (cont == 1) {
            max = alt;
            min = alt;
        } else {

            if (alt > max) {
                max = alt;
            }
            if (alt < min) {
                min = alt;
            }
        }

        printf("Ingrese otra altura (o 0 / numero negativo para salir): ");
        scanf("%f", &alt);
    }

    if (cont > 0) {
        printf("\n--- RESULTADOS ---\n");
        printf("Altura maxima: %.2f\n", max);
        printf("Altura minima: %.2f\n", min);
        printf("Media de alturas: %.2f\n", sum / cont);
    } else {
        printf("\nNo se ingresaron alturas validas.\n");
    }

    return 0;
}

```

```
Ingrese una altura (o 0 / numero negativo para salir): 6
Ingrese otra altura (o 0 / numero negativo para salir): 2165
Ingrese otra altura (o 0 / numero negativo para salir): 32
Ingrese otra altura (o 0 / numero negativo para salir): 0

--- RESULTADOS ---
Altura maxima: 2165.00
Altura minima: 6.00
Media de alturas: 734.33

Process returned 0 (0x0)   execution time : 8.655 s
Press any key to continue.
```

Problema 1.7. Raíz cuadrada entera

Desarrolla un programa que calcule la raíz cuadrada entera por defecto de un número N positivo dado, que se define como el mayor número entero R > 0 tal que $R^2 \leq N$.

Pista: Dada una variable entera i = 1, itera incrementando su valor de uno en uno mientras $i^2 \leq N$. La aproximación por defecto de la raíz es R = i − 1.

Análisis del problema

- Entrada: un numero entero positivos
- Procesos: validar el numero ingresado sea mayor que cero , si es menor o igual a cero , el programa muestra un error y no calcula nada
  inicia un contador en uno
  usa un bucle para elevar i al cuadrado en cada vuelta. Mientras ese resultado sea menor o igual a n. incrmentamos i de uno en uno
  cuando el cuadrado supera a n , el bucle se detiene. La raíz entera exacta por defecto será el paso anterior, es decir R=i-1
- Salida : la raíz cuadrade entera por defecto R

Tabla de objetos

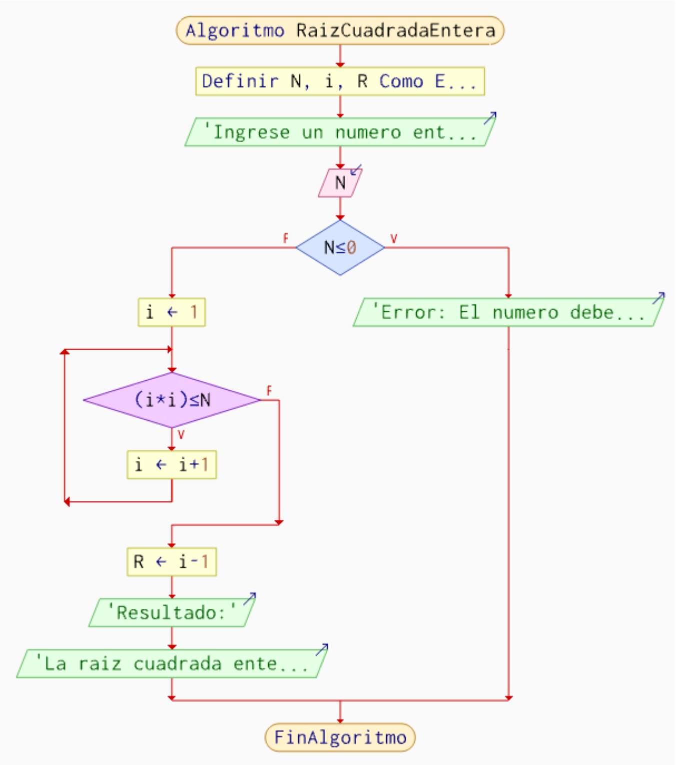
OBJETO	NOMBRE	VALOR	TIPO
Variable	N	Numero entero positivo ingresado por teclado	Entero
Variable	I	Contador	Entero
Variable	R	Resultado de la raíz cuadrada entera por defecto	Entero

Seudocódigo

```
Algoritmo RaizCuadradaEntera

    Definir N, i, R Como Entero

    Escribir "Ingrese un numero entero positivo:"

    Leer N

    Si N <= 0 Entonces

        Escribir "Error: El numero debe ser mayor a cero."

    Sino

        i <- 1

        Mientras (i * i) <= N Hacer

            i <- i + 1

        FinMientras

        R <- i - 1


        Escribir "Resultado:"
```

```
            Escribir "La raiz cuadrada entera es: ", R
        FinSi
FinAlgoritmo
```

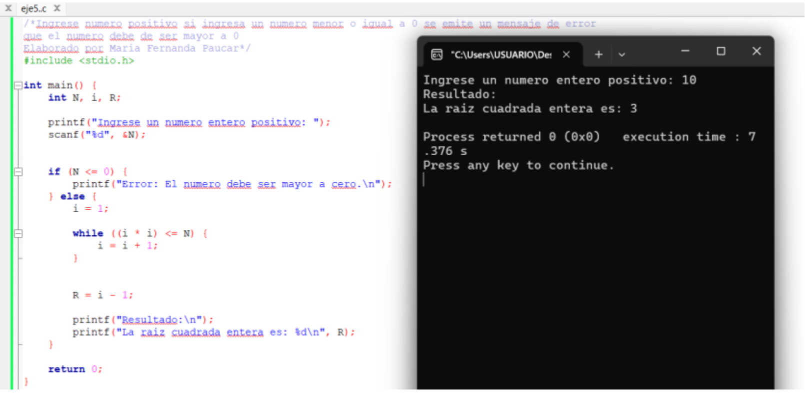
Diagrama de flujo



Algoritmo RaizCuadradaEntera
Definir N, i, R Como E...
'Ingrese un numero ent...
N
N≤0
F
V
i ← 1
'Error: El numero debe...
(i*i)≤N
F
V
i ← i+1
R ← i-1
'Resultado:'
'La raiz cuadrada ente...
FinAlgoritmo

Prueba de escritorio

PASO	N	I	R
Inicio	10	----	-----
i=1	10	1	----
Bucle 1	10	2	----
Bucle 2	10	3	------
Bucle 3	10	4	----
r= i-1	10	4	3



eje5..c
"C:\Users\USUARIO\De:

```
/*Ingrese numero positivo si ingresa un numero menor o igual a 0 se emite un mensaje de error
que el numero debe de ser mayor a 0
Elaborado por Maria Fernanda Paucar*/
#include <stdio.h>

int main() {
    int N, i, R;

    printf("Ingrese un numero entero positivo: ");
    scanf("%d", &N);


    if (N <= 0) {
        printf("Error: El numero debe ser mayor a cero.\n");
    } else {
        i = 1;

        while ((i * i) <= N) {
            i = i + 1;
        }


        R = i - 1;

        printf("Resultado:\n");
        printf("La raiz cuadrada entera es: %d\n", R);
    }

    return 0;
}
```

```
Ingrese un numero entero positivo: 10
Resultado:
La raiz cuadrada entera es: 3

Process returned 0 (0x0)   execution time : 7
.376 s
Press any key to continue.
```

Problema 1.8. Número primo

Desarrolla un programa que ingrese por teclado un número entero mayor que 1 y determine si es primo o no.

Análisis del problema

- Entrada: Un numero entero mayor que 1
- Proceso: validar que el numero sea estrictamente mayor que 1 si es menor o igual se muestra un mensaje de error.
- Un numero es pirmo si solo es divisible para 1 y para si mismo. Su comprobación tenemos que usar un bucle que divida n entre todos los números desde 2 hasta 1
  llevamos un contador de divisores cont. Usamos el operador de MOD para saber si el residuo de la división es cero si esto es verdadero significa que encontramos un numero primo y si no lo es, no es primo
- Salida: mensaje indicando que el numero si es primo o no lo es

Tabla de objetos

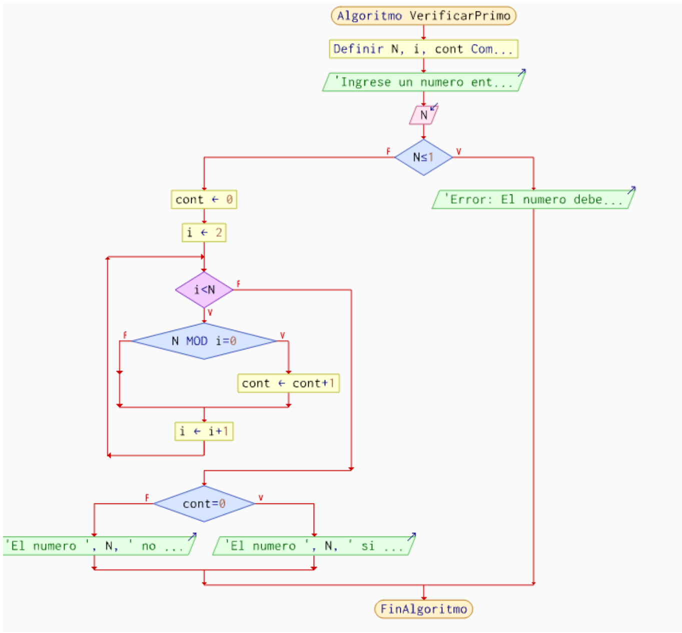
OBJETO	NOMBRE	VALOR	TIPO
Variable	N	Numero entero ingresado por el usuario	Entero
Variable	I	Variable de control para el ciclo	Entero
Variable	Cont	Contador para registrar cuantos divisores exactos tiene el numero	Entero

Seudocódigo

```
Algoritmo VerificarPrimo
Definir N, i, cont Como Entero


Escribir "Ingrese un numero entero mayor que 1:"
Leer N


Si N <= 1 Entonces
    Escribir "Error: El numero debe ser mayor que 1."
Sino
    cont <- 0
    i <- 2


    Mientras i < N Hacer
```

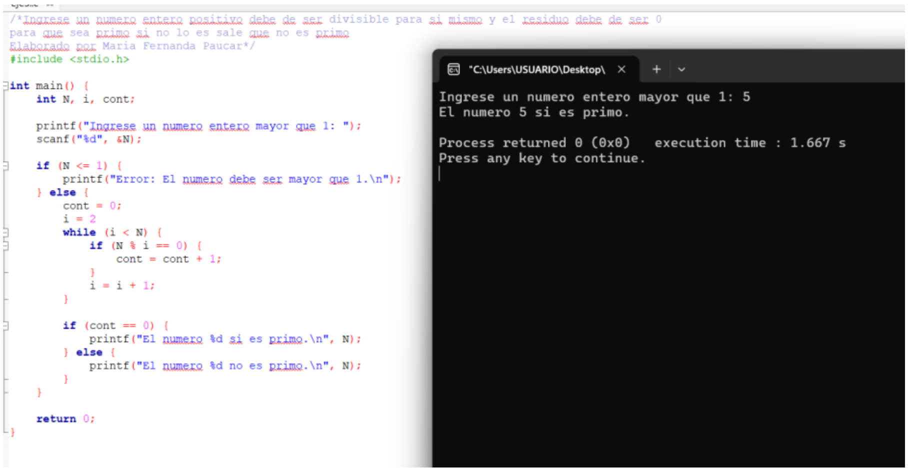
```
Si N Mod i = 0 Entonces
cont <- cont + 1
FinSi
i <- i + 1
FinMientras

Si cont = 0 Entonces
Escribir "El numero ", N, " si es primo."
Sino
Escribir "El numero ", N, " no es primo."
FinSi
FinSi
FinAlgoritmo
```

Diagrama de flujo


Algoritmo VerificarPrimo
Definir N, i, cont Com...
'Ingrese un numero ent...
N
N≤1
F
V
'Error: El numero debe...
cont ← 0
i ← 2
i<N
F
V
N MOD i=0
F
V
cont ← cont+1
i ← i+1
cont=0
F
V
'El numero ', N, ' no ...
'El numero ', N, ' si ...
FinAlgoritmo

Prueba de escritorio

PASO	N	CONT	I	N%i==0	Salida
Inicio	5	---	------		
Cont= 0 ,i=2	5	0	2	5 % 2 == 1 falso	
i++ vuelta 1	5	0	3	5 % 3 ==2 flaso	
i++ vuelta 2	5	0	4	5 % 4 == 1 falso	
i++ vuelta 3	5	0	5	Sale del ciclo porque 5<5 es falso	El numero 5 si es numero primo



```c
/*Ingrese un numero entero positivo debe de ser divisible para si mismo y el residuo debe de ser 0
para que sea primo si no lo es sale que no es primo
Elaborado por Maria Fernanda Paucar*/
#include <stdio.h>

int main() {
    int N, i, cont;

    printf("Ingrese un numero entero mayor que 1: ");
    scanf("%d", &N);

    if (N <= 1) {
        printf("Error: El numero debe ser mayor que 1.\n");
    } else {
        cont = 0;
        i = 2
        while (i < N) {
            if (N % i == 0) {
                cont = cont + 1;
            }
            i = i + 1;
        }

        if (cont == 0) {
            printf("El numero %d si es primo.\n", N);
        } else {
            printf("El numero %d no es primo.\n", N);
        }
    }

    return 0;
}
```

```
Ingrese un numero entero mayor que 1: 5
El numero 5 si es primo.

Process returned 0 (0x0)   execution time : 1.667 s
Press any key to continue.
```

Problema 1.9. Números primos comprendidos entre dos números dados

Desarrolla un programa que muestre por pantalla todos los números primos comprendidos entre dos números enteros introducidos por teclado.

- Entrada: Dos números enteros que marcan el inicio (n1) y el fin (n2) del rango.
- Proceso:El programa toma los dos números introducidos.
  Se usa un ciclo para recorrer de uno en uno todos los números que existen entre n1 y n2.
  Por cada número que va revisando dentro del rango, se descartan los menores o iguales a 1 (porque no pueden ser primos).Para los números válidos del rango, se hace una pequeña prueba dividiéndolos entre los números anteriores a ellos para saber si tienen divisores exactos. Si no tienen ningún divisor, significa que encontramos un número primo dentro del rango y se guarda.
- Salida: La lista en pantalla de todos los números que resultaron ser primos dentro de los límites dados.

Tabla de objetos

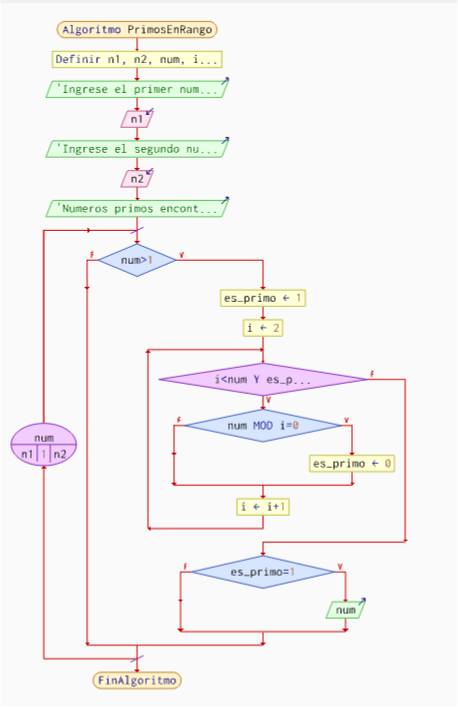
OBJETO	NOMBRE	VALOR	TIPO
Cariable	N1	Primer limte ingresado por el usuario	Entero
Variable	N2	Segundo limite ingresado por el usuario	Entero
Variable	Num	Variable para recorrer el rango de numeros	Entero
Variable	I	Variable de control para probar divisores del nuemro actual	Enteros
Variable	Es primo	Indicador para saber si el numero actual del rango cumple la condición 1=si , 0=no	Entero

Seudocódigo

```
Algoritmo PrimosEnRango
```

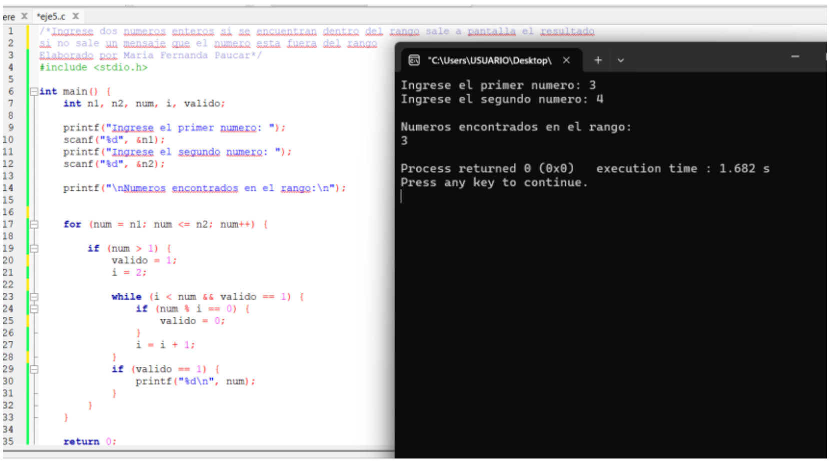
```
Definir n1, n2, num, i, es_primo Como Entero

Escribir "Ingrese el primer numero del rango:"
Leer n1
Escribir "Ingrese el segundo numero del rango:"
Leer n2

Escribir "Numeros primos encontrados en el rango:"

Para num <- n1 Hasta n2 Con Paso 1 Hacer

    Si num > 1 Entonces
        es_primo <- 1
        i <- 2

        Mientras i < num Y es_primo = 1 Hacer
            Si num Mod i = 0 Entonces
                es_primo <- 0
            FinSi
            i <- i + 1
        FinMientras

        Si es_primo = 1 Entonces
            Escribir num
        FinSi
    FinSi

FinPara
FinAlgoritmo
```

Diagrama de flujo


Algoritmo PrimosEnRango
Definir n1, n2, num, i...
'Ingrese el primer num...
n1
'Ingrese el segundo nu...
n2
'Numeros primos encont...
num>1
F
V
es_primo ← 1
i ← 2
i<num Y es_p...
F
V
num MOD i=0
F
V
es_primo ← 0
i ← i+1
es_primo=1
F
V
num
num
n1
1
n2
FinAlgoritmo

Prueba de escritorio

Num	Num>1	Divisores comprobados	Validación	Salida
3	Si	3%2! =0	1	2
4	Si	4%2==0	0	No se muestra
5	Si	5%2!=0, 5%3!=0 , 5%4!=0	1	5


ere
*eje5..c
"C:\Users\USUARIO\Desktop\

```c
/*Ingrese dos numeros enteros si se encuentran dentro del rango sale a pantalla el resultado
si no sale un mensaje que el numero esta fuera del rango
Elaborado por Maria Fernanda Paucar*/
#include <stdio.h>

int main() {
    int n1, n2, num, i, valido;

    printf("Ingrese el primer numero: ");
    scanf("%d", &n1);
    printf("Ingrese el segundo numero: ");
    scanf("%d", &n2);

    printf("\nNumeros encontrados en el rango:\n");


    for (num = n1; num <= n2; num++) {

        if (num > 1) {
            valido = 1;
            i = 2;

            while (i < num && valido == 1) {
                if (num % i == 0) {
                    valido = 0;
                }
                i = i + 1;
            }
            if (valido == 1) {
                printf("%d\n", num);
            }
        }
    }

    return 0;
```

```
Ingrese el primer numero: 3
Ingrese el segundo numero: 4

Numeros encontrados en el rango:
3

Process returned 0 (0x0)   execution time : 1.682 s
Press any key to continue.
```

Problema 1.10. Factorial

¡Desarrolla un programa que lea un número entero n ≥ 0 y calcule n! = 1 · 2 · … · n, sabiendo que 0! = 1.
Ejemplo: n = 5 ⟶ 120.
Pista: Usa un acumulador que inicie en 1 y se multiplique en cada vuelta del ciclo.
Para pensar: ¿Qué ocurre si el usuario ingresa un número negativo? ¿Y con n = 20 en una variable de tipo int?

Análisis del problema

- Entrada: Un número entero n que debe ser mayor o igual a cero (\(n \ge 0\)).
- Proceso: Validar la entrada: Si el usuario ingresa un número negativo, el programa debe mostrar un error ya que el factorial no está definido para números negativos.
  Inicializar un acumulador de multiplicaciones en uno (f <- 1). Esto cumple automáticamente con el caso especial de que \(0! = 1\).
  Si n > 0, usar un ciclo (puede ser un Para o un Mientras) que vaya desde 1 hasta n, multiplicando en cada vuelta el valor actual del acumulador por el número del contador (f <- f * i).
- Salida: El resultado del factorial (\(n!\)).

Tabla de objetos

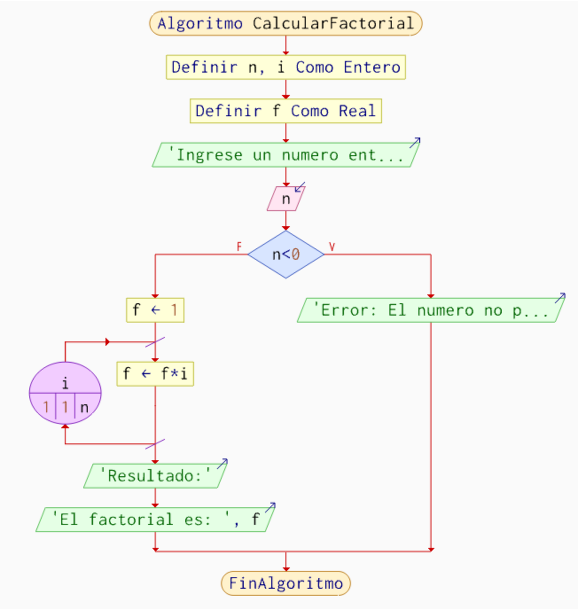
OBJETO	NOMBRE	VALOR	TIPO
Variable	N	Numero entero ingresado por el usuario	Entero
Variable	F	Acumulador donde se guarda el resultado factorial	Real
Variable	I	Contador para controlar el ciclo de multiplicaciones	Entero

Seudocódigo

```
Algoritmo CalcularFactorial
    Definir n, i Como Entero
    Definir f Como Real
    Escribir "Ingrese un numero entero (n >= 0):"
    Leer n
    Si n < 0 Entonces
        Escribir "Error: El numero no puede ser negativo."
    Sino
        f <- 1
```

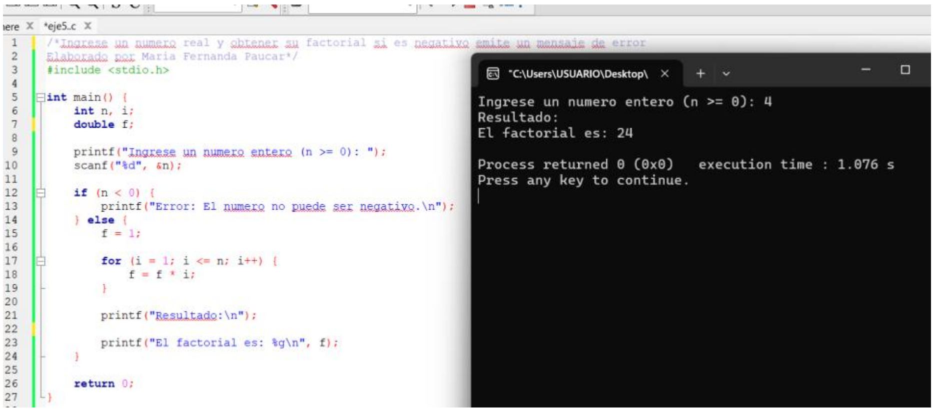
```
            Para i <- 1 Hasta n Con Paso 1 Hacer
                f <- f * i
            FinPara
            Escribir "Resultado:"
            Escribir "El factorial es: ", f
        FinSi
    FinAlgoritmo
```

Diagrama de flujo


Algoritmo CalcularFactorial
Definir n, i Como Entero
Definir f Como Real
'Ingrese un numero ent...
n
n<0
F
V
f ← 1
'Error: El numero no p...
f ← f*i
i
1
1
n
'Resultado:'
'El factorial es: ', f
FinAlgoritmo

Prueba de escritorio

PASO	N	I	F	SALIDA
Inicio	4	------	1	
Vuelta 1	4	1	1	
Vuelta 2	4	2	2	
Vuelta 3	4	3	6	
Vuelta 4	4	4	24	El factorial es : 24


ere
*eje5..c
"C:\Users\USUARIO\Desktop\

```c
/*Ingrese un numero real y obtener su factorial si es negativo emite un mensaje de error
Elaborado por Maria Fernanda Paucar*/
#include <stdio.h>

int main() {
    int n, i;
    double f;

    printf("Ingrese un numero entero (n >= 0): ");
    scanf("%d", &n);

    if (n < 0) {
        printf("Error: El numero no puede ser negativo.\n");
    } else {
        f = 1;

        for (i = 1; i <= n; i++) {
            f = f * i;
        }

        printf("Resultado:\n");

        printf("El factorial es: %g\n", f);
    }

    return 0;
}
```

```
Ingrese un numero entero (n >= 0): 4
Resultado:
El factorial es: 24

Process returned 0 (0x0)   execution time : 1.076 s
Press any key to continue.
```

Problema 1.11. Serie de Fibonacci

Desarrolla un programa que lea un entero n ≥ 1 y muestre los primeros n términos de la serie de Fibonacci (0, 1, 1, 2, 3, 5, 8, …), donde cada término es la suma de los dos anteriores. Ejemplo: n = 7 → 0 1 1 2 3 5 8.

Pista: Necesitas tres variables: el término anterior, el actual y el siguiente.

Para pensar: ¿Cómo cambia tu algoritmo si solo debes mostrar el término número n y no toda la serie?

Análisis del problema

- Entrada: Un número entero n que representa la cantidad de términos a mostrar (\(n \ge 1\)).
- Proceso: Validar la entrada: Si el usuario ingresa un número menor que 1, se muestra un mensaje de error.
  La serie siempre empieza con dos valores fijos: 0 y 1.
  Si n = 1, solo se muestra el 0. Si n = 2, se muestran 0 y 1.
  Si n >= 3, mostramos los dos primeros y usamos un ciclo desde el tercer término hasta n. En cada vuelta, calculamos el término siguiente sumando los dos anteriores (sig <- ant + act), actualizamos las variables para el siguiente paso (ant <- act, act <- sig) y mostramos el resultado.
- Salida: Los primeros n términos de la serie de Fibonacci en la pantalla.

Tabla de objetos

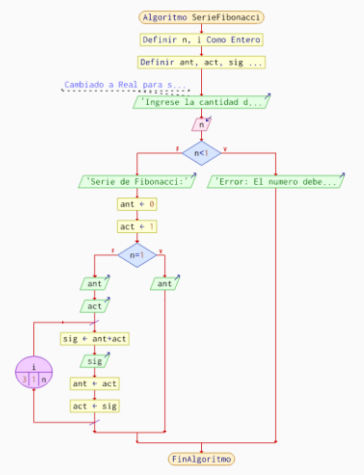
OBJETO	NOMBRE	VALOR	TIPO
Variable	N	Cantidad de términos de la seria a mostrar	Entero
Variable	Ant	Guarda el termino anterior de la serie	Entero
Variable	Act	Guarda el termino actual de la serie	Entero
Variable	Sig	Guarda el termino siguiente suma de los dos anteriores	Entero
Variable	I	Contador para controlar las vueltas del ciclo	Entero

Seudocodigo

```
Algoritmo SerieFibonacci

    Definir n, i Como Entero

    Definir ant, act, sig Como Real // Cambiado a Real para soportar numeros gigantes


    Escribir "Ingrese la cantidad de terminos (n >= 1):"

    Leer n


    Si n < 1 Entonces
```

```
        Escribir "Error: El numero debe ser mayor o igual a 1."
    Sino
        Escribir "Serie de Fibonacci:"

        ant <- 0
        act <- 1

        Si n = 1 Entonces
            Escribir ant
        Sino
            Escribir ant
            Escribir act

            Para i <- 3 Hasta n Con Paso 1 Hacer
                sig <- ant + act
                Escribir sig

                ant <- act
                act <- sig
            FinPara
        FinSi
    FinSi
FinAlgoritmo
```

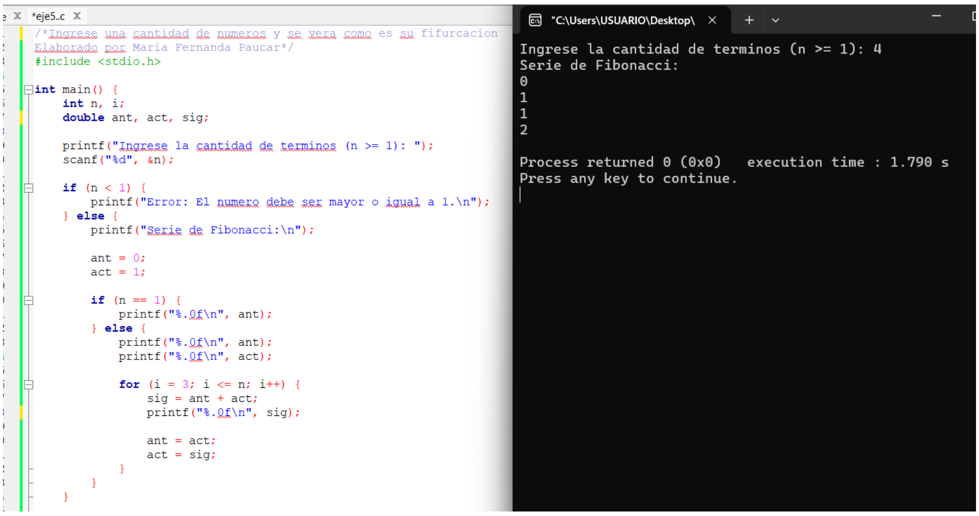
Diagrama de flujo



Algoritmo SerieFibonacci
Definir n, i Como Entero
Definir ant, act, sig ...
Cambiado a Real para s...
'Ingrese la cantidad d...
n
n<1
F
V
'Serie de Fibonacci:'
'Error: El numero debe...
ant ← 0
act ← 1
n=1
F
V
ant
ant
act
sig ← ant+act
sig
ant ← act
act ← sig
i
3
1
n
FinAlgoritmo

Prueba de escritorio

PASO	N	I	ANT	ACT	SIG
Inicio	4	-----	0	1	------
Muestra primeros	4	----	0	1	----
Vuelta i=3	4	3	1	1	1
Vuelta i=4	4	4	1	2	2
Salida					0 1 1 2



```c
/*Ingrese una cantidad de numeros y se vera como es su fifurcacion
Elaborado por Maria Fernanda Paucar*/
#include <stdio.h>

int main() {
    int n, i;
    double ant, act, sig;

    printf("Ingrese la cantidad de terminos (n >= 1): ");
    scanf("%d", &n);

    if (n < 1) {
        printf("Error: El numero debe ser mayor o igual a 1.\n");
    } else {
        printf("Serie de Fibonacci:\n");

        ant = 0;
        act = 1;

        if (n == 1) {
            printf("%.0f\n", ant);
        } else {
            printf("%.0f\n", ant);
            printf("%.0f\n", act);

            for (i = 3; i <= n; i++) {
                sig = ant + act;
                printf("%.0f\n", sig);

                ant = act;
                act = sig;
            }
        }
    }
```

```
Ingrese la cantidad de terminos (n >= 1): 4
Serie de Fibonacci:
0
1
1
2

Process returned 0 (0x0)   execution time : 1.790 s
Press any key to continue.
```

Problema 1.12. Suma de dígitos

Desarrolla un programa que lea un entero positivo y calcule la suma de sus dígitos.

Ejemplo: 4 825 ⟶ 4 + 8 + 2 + 5 = 19.

Análisis del problema

- Entrada: Un número entero positivo (num).
- Proceso: Validar que el número ingresado sea mayor o igual a cero (num >= 0).
  Inicializar un acumulador para la suma en cero (sum <- 0).
  Para extraer los dígitos uno por uno de derecha a izquierda, usamos un ciclo que se repita mientras el número sea mayor que cero (num > 0)
  En cada vuelta del ciclo: Obtenemos el último dígito con la operación módulo: dig <- num Mod 10 Sumamos ese dígito al acumulador: sum <- sum + dig.. Eliminamos el último dígito del número original mediante una división entera: num <- Trunc(num / 10).
- Slida: La suma total de los dígitos calculados.

Tabla de objetos

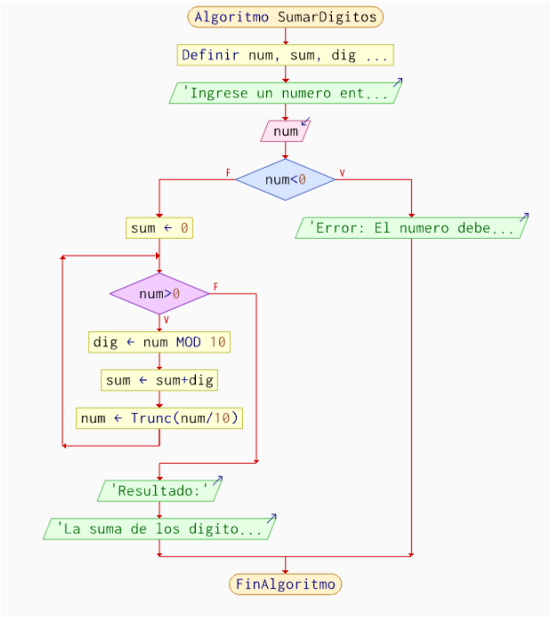
OBJETOS	NOMBRE	VALOR	TIPO
Variable	Num	Numero entero ingresado por el usuario	Entero
Variable	Sum	Acumulador donde se guarda la suma de los dígitos	Entero

Variable	Dig	Almacena temporalmente el ultimo digito extraído	Entero

Seudocodigo

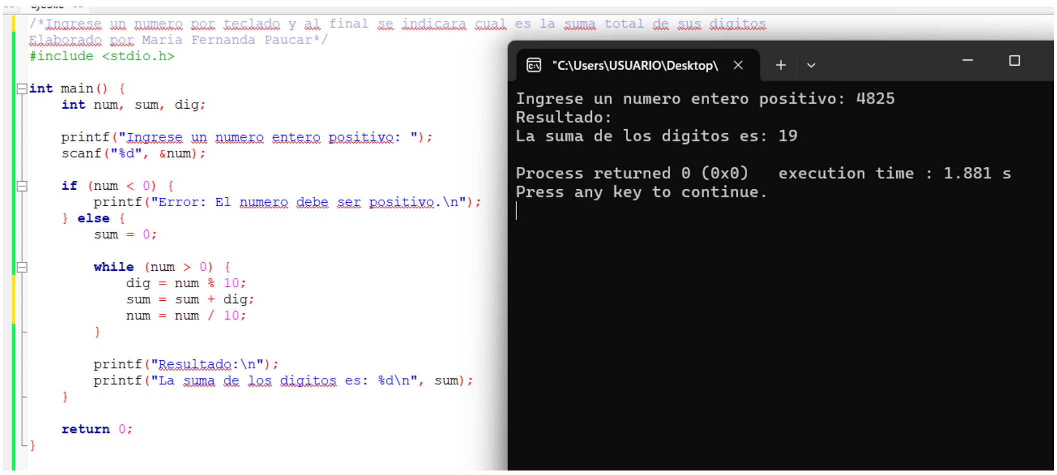
```
Algoritmo SumarDigitos
    Definir num, sum, dig Como Entero

    Escribir "Ingrese un numero entero positivo:"
    Leer num

    Si num < 0 Entonces
        Escribir "Error: El numero debe ser positivo."
    Sino
        sum <- 0

        Mientras num > 0 Hacer
            dig <- num Mod 10
            sum <- sum + dig
            num <- Trunc(num / 10)
        FinMientras

        Escribir "Resultado:"
        Escribir "La suma de los digitos es: ", sum
    FinSi
FinAlgoritmo
```

Diagrama de flujo


Algoritmo SumarDigitos
Definir num, sum, dig ...
'Ingrese un numero ent...
num
num<0
F
V
sum ← 0
'Error: El numero debe...
num>0
F
V
dig ← num MOD 10
sum ← sum+dig
num ← Trunc(num/10)
'Resultado:'
'La suma de los digito...
FinAlgoritmo

Prueba de escritorio

PASO	NUM	DIG	SIG	NOTAS
Inicio	4825	-------	0	El usuario ingresa el numero
Vuelta 1	482	5	5	4825 % 10 = 5 | sum = 0 + 5 | 4825 / 10 = 482
Vuelta 2	48	2	7	482 % 10 = 2 | sum = 5 + 2 | 482 / 10 = 48
Vuelta 3	4	8	15	48 % 10 = 8 | sum = 7 + 8 | 48 / 10 = 4
Vuelta 4	0	4	19	4 % 10 = 4 | sum = 15 + 4 | 4 / 10 = 0
Salida				La suma de los dígitos es: 19



```c
/*Ingrese un numero por teclado y al final se indicara cual es la suma total de sus digitos
Elaborado por Maria Fernanda Paucar*/
#include <stdio.h>

int main() {
    int num, sum, dig;

    printf("Ingrese un numero entero positivo: ");
    scanf("%d", &num);

    if (num < 0) {
        printf("Error: El numero debe ser positivo.\n");
    } else {
        sum = 0;

        while (num > 0) {
            dig = num % 10;
            sum = sum + dig;
            num = num / 10;
        }

        printf("Resultado:\n");
        printf("La suma de los digitos es: %d\n", sum);
    }

    return 0;
}
```

"C:\Users\USUARIO\Desktop\

```
Ingrese un numero entero positivo: 4825
Resultado:
La suma de los digitos es: 19

Process returned 0 (0x0)   execution time : 1.881 s
Press any key to continue.
```

Problema 1.13. Número capicúa

Desarrolla un programa que lea un entero positivo y determine si es capicúa, es decir, si se lee igual de izquierda a derecha que de derecha a izquierda.
Ejemplos: 12321 es capicúa; 1231 no lo es.
Pista: Construye el número invertido con el mismo recurso del ejercicio 1.12 y compáralo con el original.

Análisis del problema

- Entrada: Un número entero positivo (numero_original).
- Proceso: Guardar una copia del número original para la comparación final. Mediante un ciclo continuo, extraer el último dígito del número usando el operador residuo (MOD 10).Acumular este dígito en una variable (numero_invertido) multiplicando previamente su valor acumulado por 10 (desplazamiento posicional).Reducir el número original dividiéndolo entre 10 de forma entera (TRUNC) para procesar el siguiente dígito hasta llegar a 0.Comparar si el número invertido final es exactamente igual al número original guardado.
- Salida: Un mensaje de texto indicando si el número ingresado es capicúa o no lo es.

Tabla de objetos

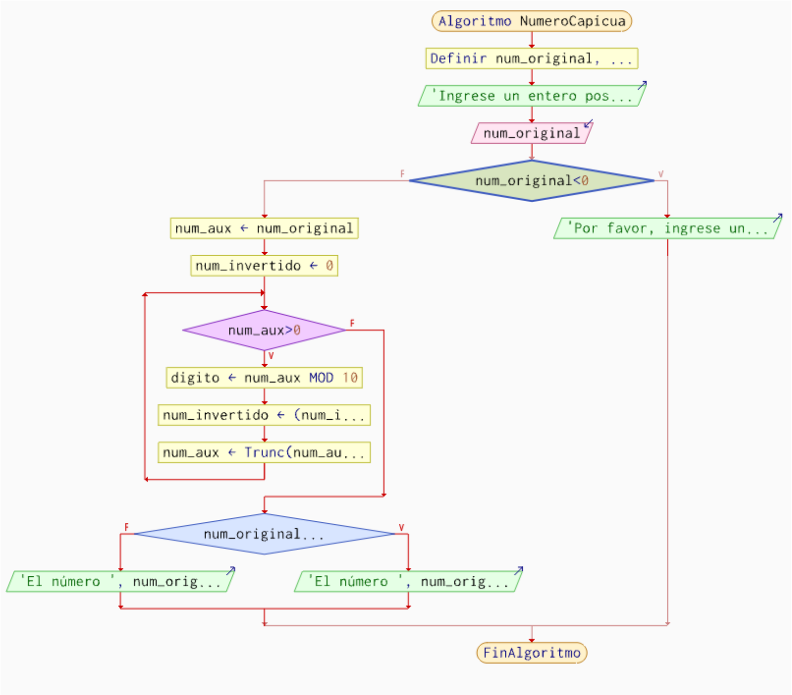
OBJETO	NOMBRE	VALOR	TIPO
Numero a evaluar	Num_original	Definido por el usuario	Entero
Copia para operar	Num_aux	Igual a num_original	Entero
Acumulador del reverso	Num_invertido	Inicializado en 0	Entero
Residuo del digito	Digito	Calculado en cada iteraccion	Entero

Seudocódigo

```
Algoritmo NumeroCapicua
  Definir num_original, num_aux, num_invertido, digito Como Entero


  Escribir "Ingrese un entero positivo:"
  Leer num_original
  Si num_original < 0 Entonces
    Escribir "Por favor, ingrese un número entero positivo válido."
  Sino
    num_aux <- num_original
    num_invertido <- 0
    Mientras num_aux > 0 Hacer
      digito <- num_aux Mod 10
      num_invertido <- (num_invertido * 10) + digito
      num_aux <- Trunc(num_aux / 10)
    FinMientras
```

```
        Si num_original = num_invertido Entonces
            Escribir "El número ", num_original, " es capicúa."
        Sino
            Escribir "El número ", num_original, " no es capicúa."
        FinSi
    FinSi
FinAlgoritmo
```

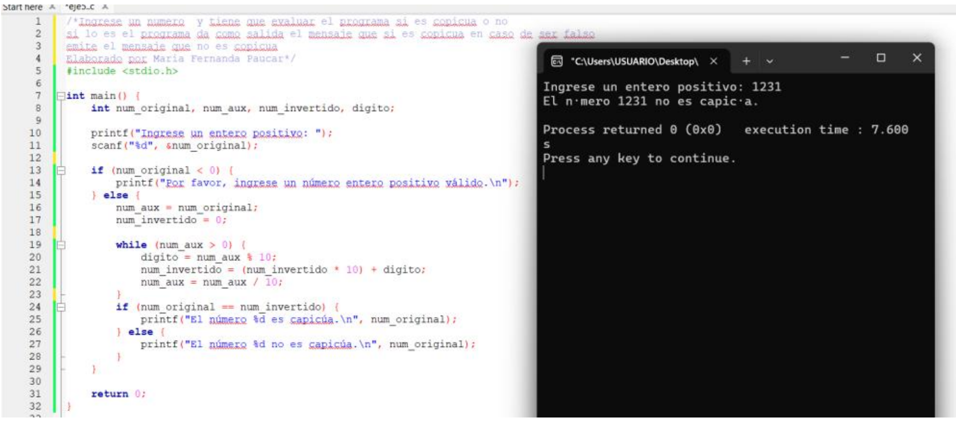
Diagrama de flujo


Algoritmo NumeroCapicua
Definir num_original, ...
'Ingrese un entero pos...
num_original
num_original<0
F
V
'Por favor, ingrese un...
num_aux ← num_original
num_invertido ← 0
num_aux>0
F
V
digito ← num_aux MOD 10
num_invertido ← (num_i...
num_aux ← Trunc(num_au...
num_original...
F
V
'El número ', num_orig...
'El número ', num_orig...
FinAlgoritmo

Prueba de escritorio

PASO	Num_original	Num_aux	Digito num_aux%10	Num invertido (num_invertido *10)+ digito	Num_aux>0?
Inicio	1231	1231	-------	0	Si entra al bucle
1	1231	123	1	(0*10)+1=1	Si
2	1231	12	3	(1 * 10) + 3 = 13	Si
3	1231	1	2	(13 * 10) + 2 = 132	Si
4	1231	0	1	(132 * 10) + 1 = 1321	No

Se evalua la condición num_original == num_invertido entonces 1231==1321 al ser falsa el programa ejecuta el bloque del else y sale a consola el mensaje “ el numero 1231 no es copicua”



```c
/*Ingrese un numero  y tiene que evaluar el programa si es copicua o no
si lo es el programa da como salida el mensaje que si es copicua en caso de ser falso
emite el mensaje que no es copicua
Elaborado por María Fernanda Paucar*/
#include <stdio.h>

int main() {
    int num_original, num_aux, num_invertido, digito;

    printf("Ingrese un entero positivo: ");
    scanf("%d", &num_original);

    if (num_original < 0) {
        printf("Por favor, ingrese un número entero positivo válido.\n");
    } else {
        num_aux = num_original;
        num_invertido = 0;

        while (num_aux > 0) {
            digito = num_aux % 10;
            num_invertido = (num_invertido * 10) + digito;
            num_aux = num_aux / 10;
        }
        if (num_original == num_invertido) {
            printf("El número %d es capicúa.\n", num_original);
        } else {
            printf("El número %d no es capicúa.\n", num_original);
        }
    }

    return 0;
}
```

```
Ingrese un entero positivo: 1231
El n·mero 1231 no es capic·a.

Process returned 0 (0x0)   execution time : 7.600
s
Press any key to continue.
```

Problema 1.14. Máximo común divisor (algoritmo de Euclides)

Desarrolla un programa que lea dos enteros positivos a y b y calcule su máximo común divisor con el algoritmo de Euclides: mientras b ≠ 0, reemplaza (a, b) por (b, a % b). Al terminar, a es el MCD. Ejemplo: MCD(48, 18) = 6.

Pista: Haz la prueba de escritorio con (48, 18): los pares sucesivos son (48, 18), (18, 12), (12, 6) y (6, 0).

Análisis del problema

- Entrada: Dos números enteros positivos que nos da el usuario (los llamaremos num1 y num2).
- Proceso:Guardamos los valores originales para no perderlos al mostrar el mensaje final.
  Usamos un ciclo que se repita mientras el segundo número no sea cero.
  Dentro del ciclo, sacamos el residuo de la división (num1 Mod num2) y lo guardamos en una variable de ayuda (residuo).
  Hacemos el intercambio que pide Euclides: el primer número toma el valor del segundo, y el segundo número toma el valor del residuo.
  Cuando el segundo número por fin llegue a cero, el ciclo se detiene. El resultado final (MCD) habrá quedado guardado en el primer número.
- Salida: Mostrar en pantalla cuál es el Máximo Común Divisor (MCD) de los dos números ingresados.

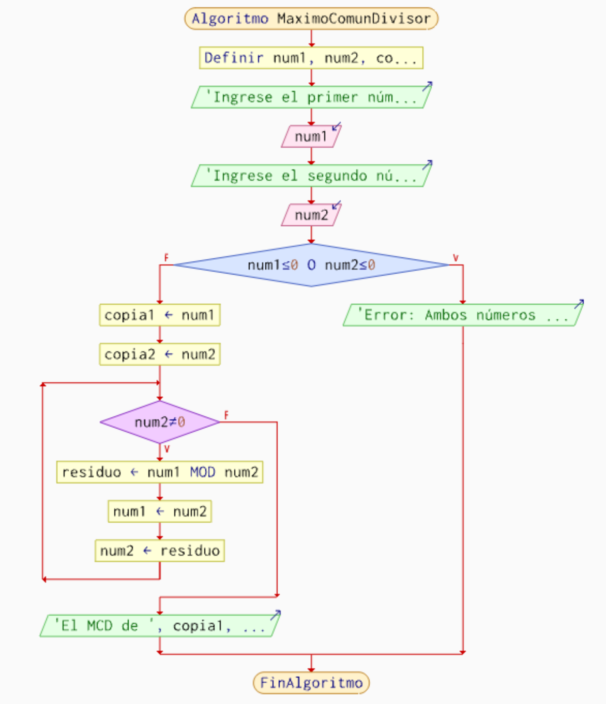
Tabla de objetos

OBJETOS	NOMBRE	VALOR	TIPO
Primer numero ingresado	Num1	Definido por el usuario	Entero
Segundo numero ingresado	Num2	Definido por el usuario	Entero
Copia del primer numero	Copia1	Igual a num1	Entero
Copiar del segundo numero	Copia2	Igual a num2	Entero
Guarda el resto de la división	Residuo	Calculado dentro del bucle	Entero

Seudocódigo

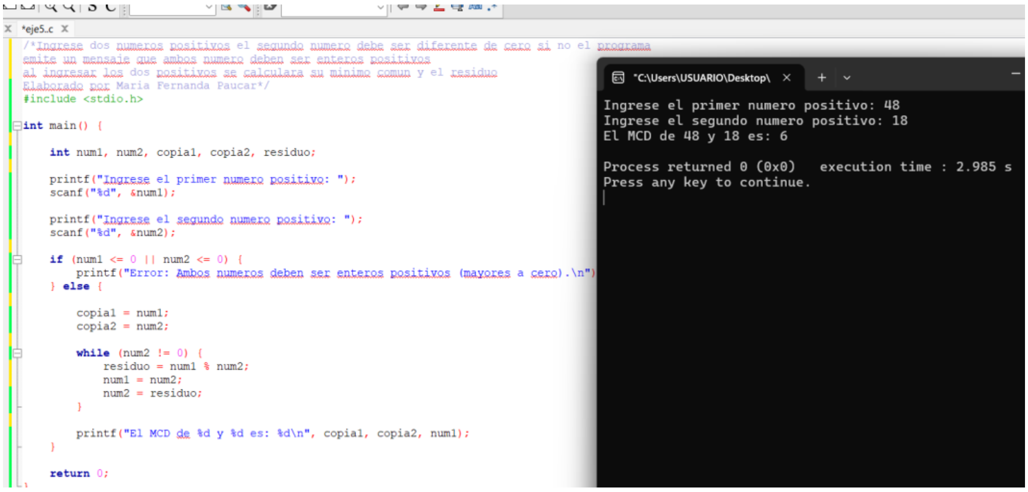
```
Algoritmo MaximoComunDivisor
    Definir num1, num2, copia1, copia2, residuo Como Entero

    Escribir "Ingrese el primer número positivo:"
    Leer num1
    Escribir "Ingrese el segundo número positivo:"
    Leer num2
    Si num1 <= 0 O num2 <= 0 Entonces
        Escribir "Error: Ambos números deben ser enteros positivos (mayores a cero)."
    Sino
        copia1 <- num1
        copia2 <- num2

        Mientras num2 <> 0 Hacer
            residuo <- num1 Mod num2
            num1 <- num2
            num2 <- residuo
        FinMientras

        Escribir "El MCD de ", copia1, " y ", copia2, " es: ", num1
    FinSi
FinAlgoritmo
```

Diagrama de flujo


Algoritmo MaximoComunDivisor
Definir num1, num2, co...
'Ingrese el primer núm...
num1
'Ingrese el segundo nú...
num2
F
num1≤0 O num2≤0
V
copia1 ← num1
'Error: Ambos números ...
copia2 ← num2
num2≠0
F
V
residuo ← num1 MOD num2
num1 ← num2
num2 ← residuo
'El MCD de ', copia1, ...
FinAlgoritmo

Prueba de escritorio

PASO	Num1	Num2	Residuo (num1%num2)	Num2!=0	Salida
Inicio	48	18	-------	Si	Entra al bucle porque 18 no es cero
1	18	12	48%12=12	Si	Num1 toma el 18 y num2 toma el residuo
2	12	6	18%12=6	Si	Num1 toma el 12 y num2 toma residuo el 6
3	6	0	12%6=0	No	Num1 toma el 6 y num2 toma el residuo 0. El bucle termina



```c
/*Ingrese dos numeros positivos el segundo numero debe ser diferente de cero si no el programa
emite un mensaje que ambos numero deben ser enteros positivos
al ingresar los dos positivos se calculara su minimo comun y el residuo
Elaborado por Maria Fernanda Paucar*/
#include <stdio.h>

int main() {

    int num1, num2, copia1, copia2, residuo;

    printf("Ingrese el primer numero positivo: ");
    scanf("%d", &num1);

    printf("Ingrese el segundo numero positivo: ");
    scanf("%d", &num2);

    if (num1 <= 0 || num2 <= 0) {
        printf("Error: Ambos numeros deben ser enteros positivos (mayores a cero).\n")
    } else {

        copia1 = num1;
        copia2 = num2;

        while (num2 != 0) {
            residuo = num1 % num2;
            num1 = num2;
            num2 = residuo;
        }

        printf("El MCD de %d y %d es: %d\n", copia1, copia2, num1);
    }

    return 0;
}
```

```
Ingrese el primer numero positivo: 48
Ingrese el segundo numero positivo: 18
El MCD de 48 y 18 es: 6

Process returned 0 (0x0)   execution time : 2.985 s
Press any key to continue.
```

Problema 1.15. Año bisiesto

Desarrolla un programa que lea un año y determine si es bisiesto. Un año es bisiesto si es divisible entre 4, excepto los años divisibles entre 100, que solo lo son si también son divisibles entre 400.

Ejemplos: 2024 sí; 1900 no; 2000 sí.

Pista: Combina los operadores &&, || y % en una sola condición, o utiliza condiciones anidadas.

Análisis del problema

- Entrada: Un año entero positivo (anio).
- Proceso:Un año es bisiesto si cumple con una regla matemática de descarte:
  Primero, verificamos si el año es divisible entre 4 (anio Mod 4 = 0). Si no lo es, no es bisiesto.
  Si sí es divisible entre 4, hay una trampa: no debe ser divisible entre 100 (anio Mod 100 = 0), a menos que también sea divisible entre 400 (anio Mod 400 = 0).Por lo tanto, si es divisible entre 4 pero no entre 100, es bisiesto. Si es divisible entre 4 y entre 100, solo es bisiesto si además es divisible entre 400.
- Salida: Un mensaje indicando si el año ingresado es bisiesto o no lo es.}

Tabla de objetos

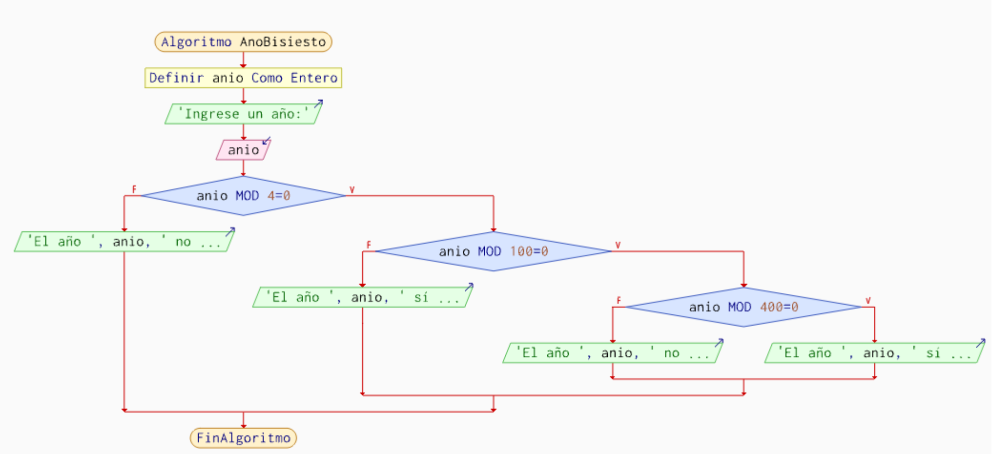
OBJETO	NOMBRE	VALOR	TIPO
Año por evaluar	Año	Definido por el usuario	entero

Seudocódigo

```
Algoritmo AnoBisiesto


 Definir anio Como Entero


 Escribir "Ingrese un año:"

 Leer anio


 Si anio Mod 4 = 0 Entonces

   Si anio Mod 100 = 0 Entonces

     Si anio Mod 400 = 0 Entonces

       Escribir "El año ", anio, " sí es bisiesto."

     Sino

       Escribir "El año ", anio, " no es bisiesto."

     FinSi

   Sino

     Escribir "El año ", anio, " sí es bisiesto."

   FinSi

 Sino
```

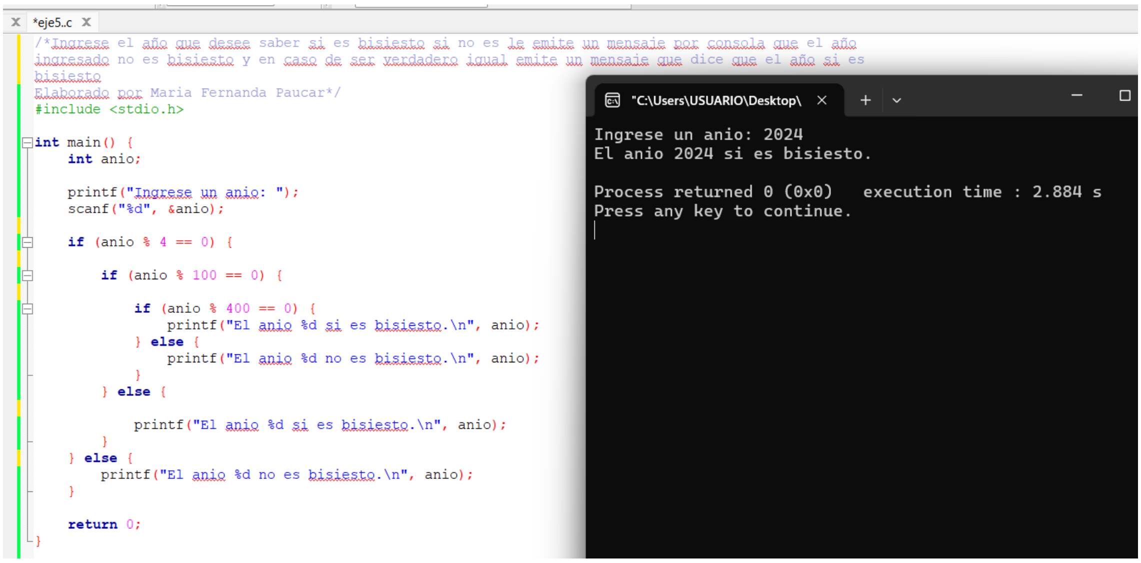
```
        Escribir "El año ", anio, " no es bisiesto."
    FinSi
FinAlgoritmo
```

Diagrama flujo


Algoritmo AnoBisiesto
Definir anio Como Entero
'Ingrese un año:'
anio
anio MOD 4=0
F
V
'El año ', anio, ' no ...
anio MOD 100=0
F
V
'El año ', anio, ' si ...
anio MOD 400=0
F
V
'El año ', anio, ' no ...
'El año ', anio, ' si ...
FinAlgoritmo

Prueba de escritorio

Año	Año%4==0	Año%100==0	Año%400==0	Salida
2024	Si	No	-----	Ingresa al primer if luego va al else de los 100 entonces el anio 2024 si es bisiesto
1900	Si	Si	No	El anio 1900
2000	Si	Si	Si	El anio 2000 si es bisiesto


*eje5..c
"C:\Users\USUARIO\Desktop\

```c
/*Ingrese el año que desee saber si es bisiesto si no es le emite un mensaje por consola que el año
ingresado no es bisiesto y en caso de ser verdadero igual emite un mensaje que dice que el año si es
bisiesto
Elaborado por Maria Fernanda Paucar*/
#include <stdio.h>

int main() {
    int anio;

    printf("Ingrese un anio: ");
    scanf("%d", &anio);

    if (anio % 4 == 0) {

        if (anio % 100 == 0) {

            if (anio % 400 == 0) {
                printf("El anio %d si es bisiesto.\n", anio);
            } else {
                printf("El anio %d no es bisiesto.\n", anio);
            }
        } else {

            printf("El anio %d si es bisiesto.\n", anio);
        }
    } else {
        printf("El anio %d no es bisiesto.\n", anio);
    }

    return 0;
}
```

```
Ingrese un anio: 2024
El anio 2024 si es bisiesto.

Process returned 0 (0x0)   execution time : 2.884 s
Press any key to continue.
```

Problema 1.16. Número perfecto

Desarrolla un programa que lea un entero positivo y determine si es perfecto, es decir, si es igual a la suma de sus divisores propios (los menores que él).
Ejemplo: 28 = 1 + 2 + 4 + 7 + 14, por lo tanto es perfecto.
Pista: Recorre los posibles divisores desde 1 hasta n − 1 y acumula los que cumplan n % d == 0.

Análisis del problema

- Entrada: Un número entero positivo (num).
- Proceso: Validar que el número ingresado sea mayor a cero. Inicializar un acumulador (suma_divisores) en 0.Crear un bucle (ciclo Para) que vaya desde 1 hasta el número anterior al ingresado (num - 1). Este contador será nuestro divisor a probar.En cada vuelta del ciclo, verificar si el número es divisible exactamente entre el contador usando el operador residuo (num Mod divisor = 0).Si el residuo es cero, significa que es un divisor propio, por lo tanto lo sumamos al acumulador (suma_divisores <- suma_divisores + divisor). Al terminar el ciclo, comparar si la suma acumulada es exactamente igual al número original.
- Salida: Un mensaje indicando si el número es perfecto o no lo es.

Tabla de objetos

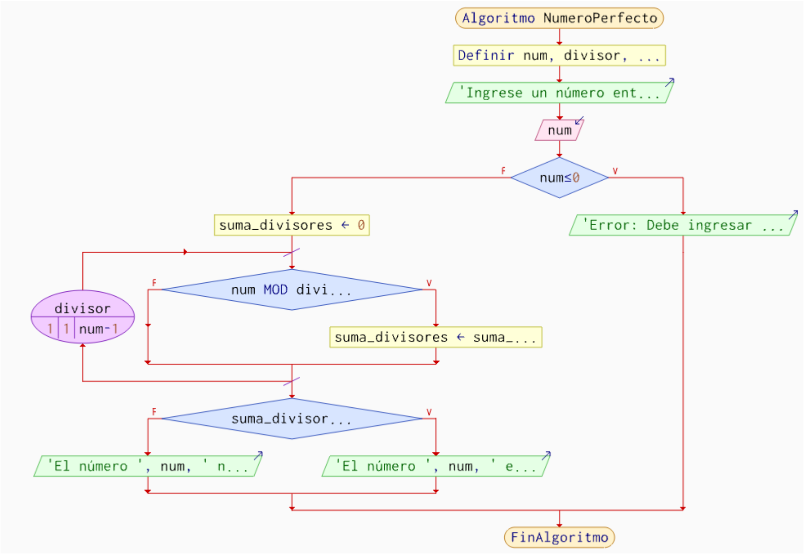
OBJETO	NOMBRE	VALOR	TIPO
Numero a evaluar	Num	Definido por el usuario	Entero
Contador del ciclo	Divisor	Variable desde 1 hasta num -1	Entero
Acumulador de la suma	Suma_divisores	Inicializado en 0	Entero

Seudocódigo

```
Algoritmo NumeroPerfecto
  Definir num, divisor, suma_divisores Como Entero


  Escribir "Ingrese un número entero positivo:"
  Leer num
  Si num <= 0 Entonces
    Escribir "Error: Debe ingresar un número mayor a cero."
  Sino
    suma_divisores <- 0


    Para divisor <- 1 Hasta num - 1 Con Paso 1 Hacer
      Si num Mod divisor = 0 Entonces
```

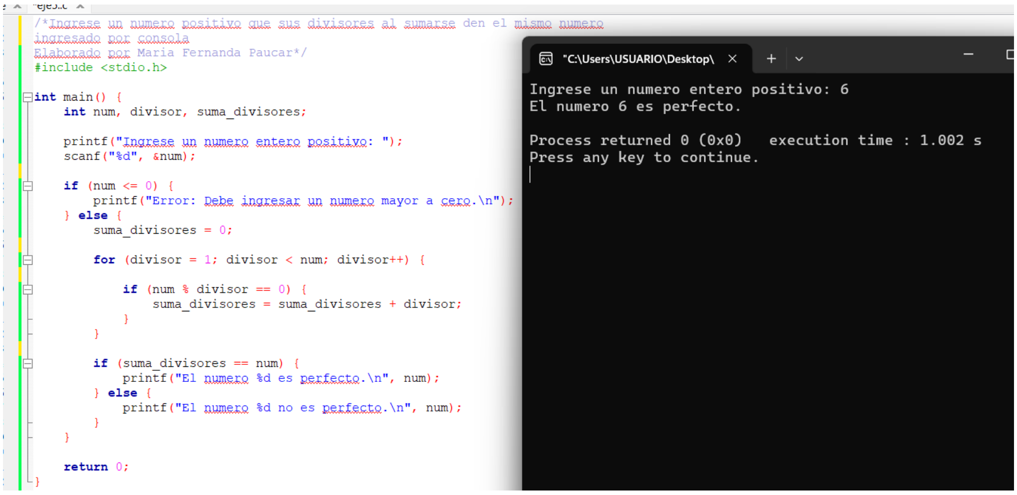
```
                suma_divisores <- suma_divisores + divisor
            FinSi
        FinPara

        Si suma_divisores = num Entonces
            Escribir "El número ", num, " es perfecto."
        Sino
            Escribir "El número ", num, " no es perfecto."
        FinSi
    FinSi
FinAlgoritmo
```

Diagrama de flujo


Algoritmo NumeroPerfecto
Definir num, divisor, ...
'Ingrese un número ent...
num
num≤0
F
V
suma_divisores ← 0
'Error: Debe ingresar ...
divisor
1
1
num-1
num MOD divi...
F
V
suma_divisores ← suma_...
suma_divisor...
F
V
'El número ', num, ' n...
'El número ', num, ' e...
FinAlgoritmo

Prueba de escritorio

Divisores	Sdivide exacto a 4!? (4%divisor==0)	Suma_divisores
1	Si	0+1=1
2	Si	1+2=3
3	No	Se queda en 3



```c
/*Ingrese un numero positivo que sus divisores al sumarse den el mismo numero
ingresado por consola
Elaborado por Maria Fernanda Paucar*/
#include <stdio.h>

int main() {
    int num, divisor, suma_divisores;

    printf("Ingrese un numero entero positivo: ");
    scanf("%d", &num);

    if (num <= 0) {
        printf("Error: Debe ingresar un numero mayor a cero.\n");
    } else {
        suma_divisores = 0;

        for (divisor = 1; divisor < num; divisor++) {

            if (num % divisor == 0) {
                suma_divisores = suma_divisores + divisor;
            }
        }

        if (suma_divisores == num) {
            printf("El numero %d es perfecto.\n", num);
        } else {
            printf("El numero %d no es perfecto.\n", num);
        }
    }

    return 0;
}
```

```
Ingrese un numero entero positivo: 6
El numero 6 es perfecto.

Process returned 0 (0x0)   execution time : 1.002 s
Press any key to continue.
```

Problema 1.17. Conversión de decimal a binario

Desarrolla un programa que lea un entero positivo en base 10 y muestre su representación en binario.

Ejemplo: 13 → 1101.

Pista: Divide entre 2 sucesivamente y guarda los restos; el resultado se lee de abajo hacia arriba.

- Análisis del problema
- Entrada: Un número entero positivo en base 10 (num_decimal).
- Proceso:Para convertir un número a binario de forma matemática, se divide sucesivamente entre 2 y se guardan los residuos (que serán solo 0 o 1). Como los residuos se leen "de abajo hacia arriba", en programación podemos usar un truco matemático muy ingenioso para construir el número al derecho: Validar que el número ingresado sea mayor o igual a cero.Usar un acumulador (num_binario) inicializado en 0 y una variable multiplicadora (posicion) inicializada en 1 (que representa las unidades, decenas, centenas, etc.). Crear un ciclo que se repita mientras el número decimal sea mayor que cero.En cada vuelta, sacar el residuo de la división entre 2 (num_decimal Mod 2) para obtener el dígito binario.Multiplicar ese dígito por nuestra variable posicion y sumarlo al acumulador num_binario. Esto hace que los nuevos residuos se vayan colocando automáticamente a la izquierda del número.Actualizar el número decimal dividiéndolo de forma entera entre 2 (Trunc(num_decimal / 2)).Multiplicar posicion por 10 para prepararse para la siguiente columna (pasar de unidades a decenas, luego centenas, etc.).
- Salida: Mostrar en pantalla el número final guardado en el acumulador (num_binario).

Tabla de objetos

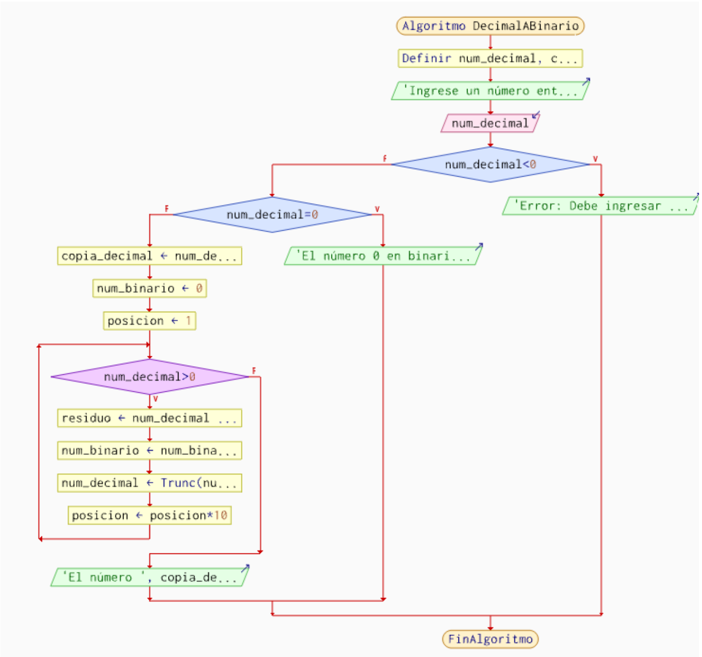
OBJETO	NOMBRE	VALOR	TIPO
Numero decimal a convertir	Num_decimal	Definido por el usuario	Entero
Copia para el mensaje final	Copia_decimal	Igual a num_decimal	Entero
Acumulador del numero binario	Num_binario	Inicializado en 0	Entero

Residuo de la división	Residuo	Calculado en cada iteración	Entero
Controlador de la posición digital	Posición	Inicalizado en 1 multiplica por 10	Entero

Seudocódigo

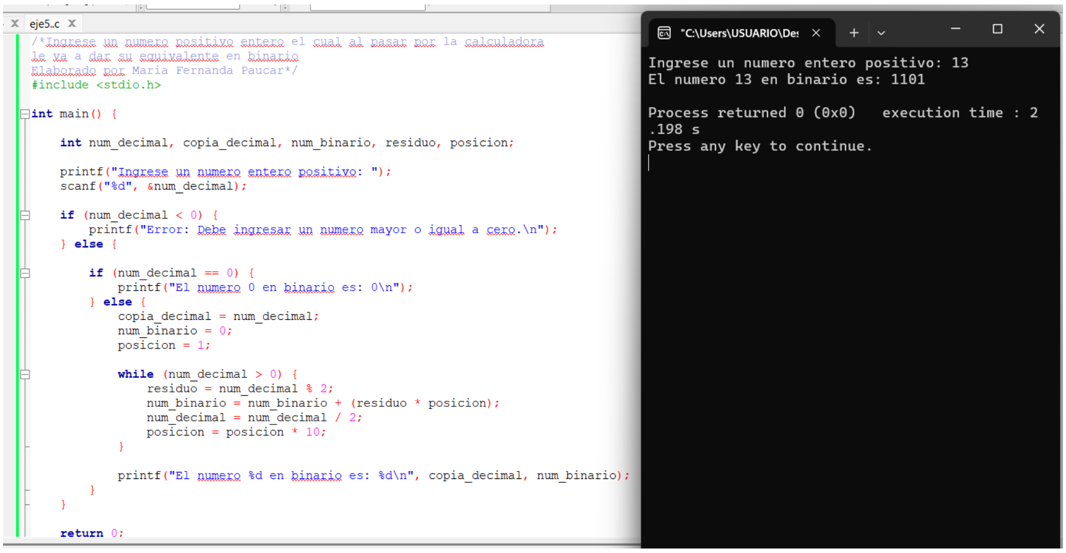
```
Algoritmo DecimalABinario

  Definir num_decimal, copia_decimal, num_binario, residuo, posicion Como Entero
           Escribir "Ingrese un número entero positivo:"
  Leer num_decimal

  Si num_decimal < 0 Entonces
    Escribir "Error: Debe ingresar un número mayor o igual a cero."
  Sino

    Si num_decimal = 0 Entonces
      Escribir "El número 0 en binario es: 0"
    Sino
      copia_decimal <- num_decimal
      num_binario <- 0
      posicion <- 1

      Mientras num_decimal > 0 Hacer
        residuo <- num_decimal Mod 2
        num_binario <- num_binario + (residuo * posicion)
        num_decimal <- Trunc(num_decimal / 2)
        posicion <- posicion * 10
      FinMientras
      Escribir "El número ", copia_decimal, " en binario es: ", num_binario
    FinSi
  FinSi
FinAlgoritmo
```

Diagrama de flujo


Algoritmo DecimalABinario
Definir num_decimal, c...
'Ingrese un número ent...
num_decimal
num_decimal<0
F
V
'Error: Debe ingresar ...
num_decimal=0
F
V
'El número 0 en binari...
copia_decimal ← num_de...
num_binario ← 0
posicion ← 1
num_decimal>0
F
V
residuo ← num_decimal ...
num_binario ← num_bina...
num_decimal ← Trunc(nu...
posicion ← posicion*10
'El número ', copia_de...
FinAlgoritmo

Prueba de escritorio

PASO	Num_decimal	Residuo (num_decimal%2)	Num_binario (acumulador)	Posición	Num_decimal>0 ?
Inicio	13	-------	0	1	Si
1	6	1	0+(1*1)=1	10	Si
2	3	0	1+(0*10)=1	100	Si
3	1	1	1+(1*100)=101	1000	Si
4	0	1	101+(1*1000)=1101	10000	No termina el bucle
Salida					El numero 13 en binario es:1101


eje5..c
"C:\Users\USUARIO\De:

```c
/*Ingrese un numero positivo entero el cual al pasar por la calculadora
le va a dar su equivalente en binario
Elaborado por Maria Fernanda Paucar*/
#include <stdio.h>

int main() {

    int num_decimal, copia_decimal, num_binario, residuo, posicion;

    printf("Ingrese un numero entero positivo: ");
    scanf("%d", &num_decimal);

    if (num_decimal < 0) {
        printf("Error: Debe ingresar un numero mayor o igual a cero.\n");
    } else {

        if (num_decimal == 0) {
            printf("El numero 0 en binario es: 0\n");
        } else {
            copia_decimal = num_decimal;
            num_binario = 0;
            posicion = 1;

            while (num_decimal > 0) {
                residuo = num_decimal % 2;
                num_binario = num_binario + (residuo * posicion);
                num_decimal = num_decimal / 2;
                posicion = posicion * 10;
            }

            printf("El numero %d en binario es: %d\n", copia_decimal, num_binario);
        }
    }

    return 0;
```

```
Ingrese un numero entero positivo: 13
El numero 13 en binario es: 1101

Process returned 0 (0x0)   execution time : 2
.198 s
Press any key to continue.
```

Problema 1.18. Triángulo de asteriscos

Desarrolla un programa que lea un entero n ≥ 1 y dibuje por pantalla un triángulo rectángulo de n filas, donde la fila i tiene i asteriscos.
Ejemplo con n = 4:

```
*
**
***
****
```

Pista: Usa un ciclo dentro de otro: el externo controla las filas y el interno, los asteriscos de cada fila.

- Análisis del problema
- Entrada: Un número entero positivo que representa la cantidad de filas (n).
- Proceso: Para dibujar este triángulo, necesitamos controlar dos cosas: en qué fila estamos y cuántos asteriscos debemos pintar en esa fila. Validar que el número de filas ingresado sea mayor o igual a 1 (n >= 1). Usar un ciclo externo (un ciclo Para controlado por la variable fila) que vaya desde 1 hasta n. Este ciclo se encarga de cambiar de renglón. Dentro de ese ciclo, colocar un ciclo interno (otro ciclo Para controlado por la variable columna) que vaya desde 1 hasta el valor actual de fila. En cada vuelta del ciclo interno, imprimir un solo asterisco (*) sin saltar de línea. Esto asegura que si estamos en la fila 3, se pinten exactamente 3 asteriscos seguidos. Al salir del ciclo interno (cuando se terminen de pintar los asteriscos de esa fila), hacer un salto de línea vacío para pasar a la siguiente fila.
- Salida: El triángulo de asteriscos dibujado en la pantalla.

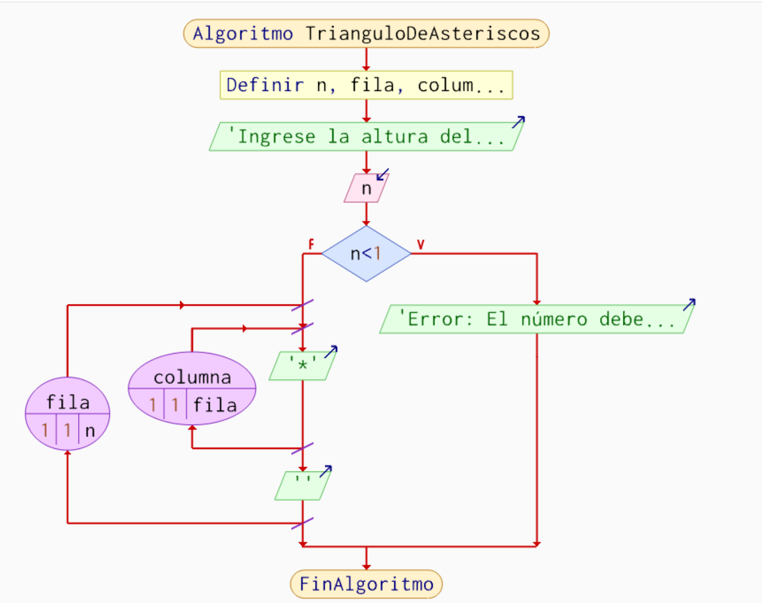
Tabla de objetos

OBJETO	NOMBRE	VALOR	TIPO
Numero de filas totales	N	Definido por el usuario	Entero
Contador de fila actual	Fila	Varia desde el 1 hasta n	ENTERO
Contador de asteriscos	Columna	Varia de 1 hasta el valor de la fila	Entero

Seudocódigo

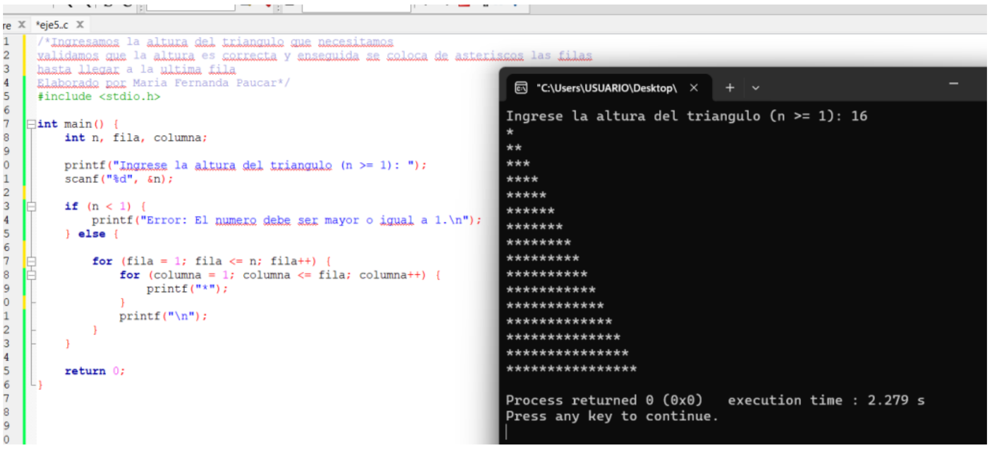
```
Algoritmo TrianguloDeAsteriscos


  Definir n, fila, columna Como Entero


  Escribir "Ingrese la altura del triángulo (n >= 1):"

  Leer n


  Si n < 1 Entonces

    Escribir "Error: El número debe ser mayor o igual a 1."
  Sino

    Para fila <- 1 Hasta n Con Paso 1 Hacer

      Para columna <- 1 Hasta fila Con Paso 1 Hacer
        Escribir "*" Sin Saltar
      FinPara

      Escribir ""

    FinPara
  FinSi
FinAlgoritmo
```

Diagrama de flujo


Algoritmo TrianguloDeAsteriscos
Definir n, fila, colum...
'Ingrese la altura del...
n
F
n<1
V
'Error: El número debe...
fila
1 1 n
columna
1 1 fila
'*'
''
FinAlgoritmo

Prueba de escritorio

Fila	Rango de asteriscos	Lo que se dibuja en pantalla
1	Va de 1 a 1	*
2	Va de 1 a 2	**
3	Va de 1 a 3	***



```c
/*Ingresamos la altura del triangulo que necesitamos
validamos que la altura es correcta y enseguida se coloca de asteriscos las filas
hasta llegar a la ultima fila
Elaborado por Maria Fernanda Paucar*/
#include <stdio.h>

int main() {
    int n, fila, columna;

    printf("Ingrese la altura del triangulo (n >= 1): ");
    scanf("%d", &n);

    if (n < 1) {
        printf("Error: El numero debe ser mayor o igual a 1.\n");
    } else {

        for (fila = 1; fila <= n; fila++) {
            for (columna = 1; columna <= fila; columna++) {
                printf("*");
            }
            printf("\n");
        }
    }

    return 0;
}
```

```
Ingrese la altura del triangulo (n >= 1): 16
*
**
***
****
*****
******
*******
********
*********
**********
***********
************
*************
**************
***************
****************

Process returned 0 (0x0)   execution time : 2.279 s
Press any key to continue.
```

Problema 1.19. Segundo mayor de una secuencia

Desarrolla un programa que lea números enteros positivos hasta que se introduzca un 0, y muestre el mayor y el segundo mayor de los valores ingresados. Si hay menos de dos valores, debe informarlo.
Ejemplo: 7, 3, 9, 5, 0 → mayor = 9, segundo mayor = 7.
Pista: Mantén dos variables, mayor y segundo, y actualízalas con cuidado cuando llegue un valor nuevo.

Análisis del problema

- Entrada: Una lista o secuencia de números enteros positivos ingresados uno por uno. El proceso se detiene cuando el usuario escribe el número 0.
- Proceso: Necesitamos contar cuántos números válidos (mayores a cero) ingresa el usuario usando un contador.Mantendremos dos variables de control: mayor y segundo_mayor, ambas inicializadas en 0. Usaremos un ciclo que pida números continuamente mientras el número ingresado no sea 0.
- Cada vez que entre un número válido:Aumentamos el contador en 1.
    - Caso A: Si el nuevo número es más grande que nuestro mayor actual, entonces el que antes era el mayor pasa a convertirse en el segundo_mayor, y el nuevo número toma el puesto del mayor.
    - Caso B: Si el número no superó al mayor, pero sí es más grande que el segundo_mayor actual, entonces simplemente actualizamos el segundo_mayor con este nuevo número.Al salir del ciclo (cuando pongan el 0), revisamos el contador. Si es menor a 2, significa que no se ingresaron suficientes datos para calcular un segundo mayor.
- Salida: Si ingresó 2 o más números, mostrar el mayor y el segundo_mayor. De lo contrario, mostrar un mensaje de aviso diciendo que faltaron datos.

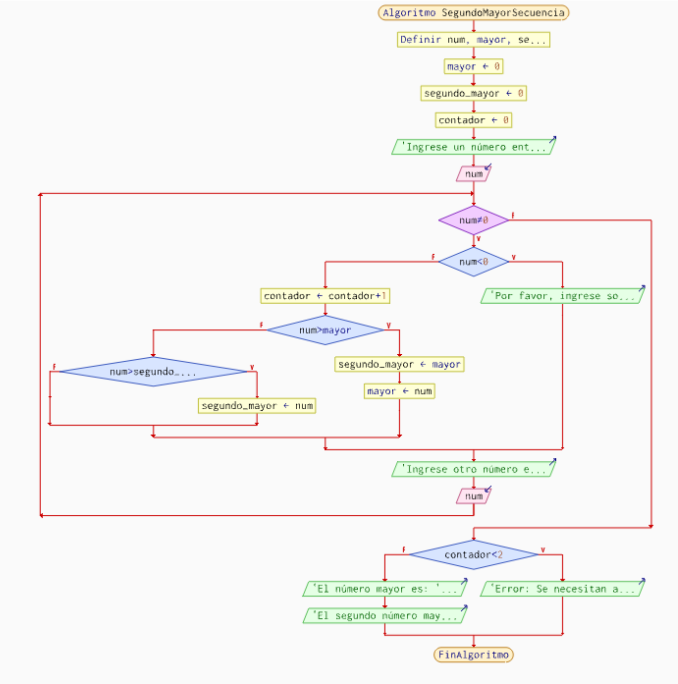
Tabla de objetos

OBJETO	NOMBRE	VALOR	TIPO
Numero leído actual	Num	Definido por el usuario en cada paso	Entero
Numero mas grande	Mayor	Inicializado en 0	Entero
El segundo mas grande	Segundo_mayor	Inicializado en 0	Entero
Contador de números	Contador	Inicializado en 0 (cuenta los validos)	Entero

Seudocódigo

```
Algoritmo SegundoMayorSecuencia

    Definir num, mayor, segundo_mayor, contador Como Entero
    mayor <- 0
    segundo_mayor <- 0
    contador <- 0
    Escribir "Ingrese un número entero positivo (0 para terminar):"
    Leer num
    Mientras num <> 0 Hacer
        Si num < 0 Entonces
            Escribir "Por favor, ingrese solo números positivos."
        Sino
            contador <- contador + 1
            Si num > mayor Entonces
                segundo_mayor <- mayor
                mayor <- num
            Sino
                Si num > segundo_mayor Entonces
                    segundo_mayor <- num
                FinSi
            FinSi
        FinSi
        Escribir "Ingrese otro número entero positivo (0 para terminar):"
        Leer num
    FinMientras
    Si contador < 2 Entonces
        Escribir "Error: Se necesitan al menos dos valores positivos para calcular el segundo mayor."
    Sino
        Escribir "El número mayor es: ", mayor
        Escribir "El segundo número mayor es: ", segundo_mayor
    FinSi
FinAlgoritmo
```

Diagrama de flujo

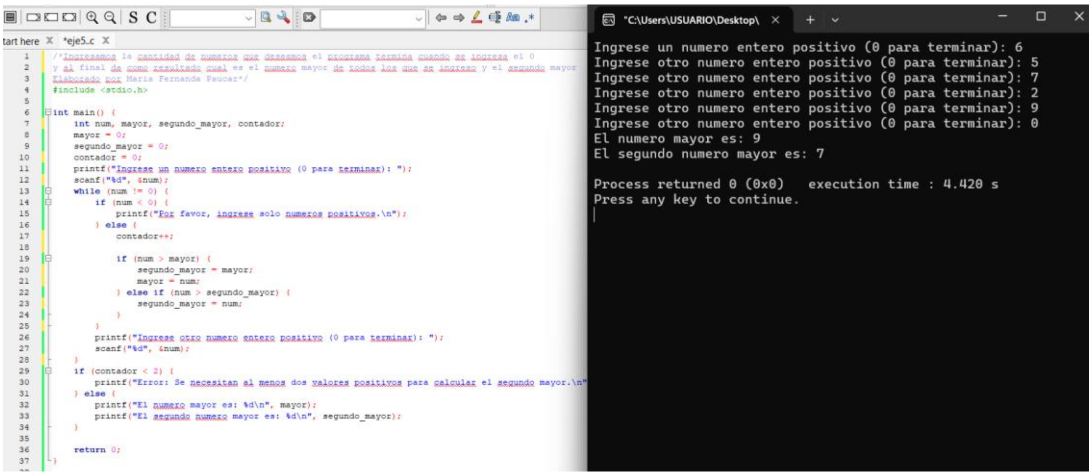

Algoritmo SegundoMayorSecuencia
Definir num, mayor, se...
mayor ← 0
segundo_mayor ← 0
contador ← 0
'Ingrese un número ent...
num
num≠0
F
V
num<0
F
V
contador ← contador+1
'Por favor, ingrese so...
num>mayor
F
V
num>segundo_....
F
V
segundo_mayor ← mayor
mayor ← num
segundo_mayor ← num
'Ingrese otro número e...
num
contador<2
F
V
'El número mayor es: '...
'Error: Se necesitan a...
'El segundo número may...
FinAlgoritmo

Prueba de escritorio

Numero ingreso	Num>mayor	Num>segundo mayor	Nuevo mayor	Nuvo segundo_mayor	Contador
7	Sí ((7 > 0)	-------	7	0	1
3	No ((3 > 7) es falso)	Sí (3 > 0)	7	3	2
9	Si(9>7)	----	9	7	3
5	No (5>9)	No (5>7 falso)	9	7	4
0	----	-----	-----	-----	Termina

Salida: el numero mayor es :9

el segundo mayor es :7


tart here
*eje5.c
"C:\Users\USUARIO\Desktop\

```c
/*Ingresamos la cantidad de numeros que deseamos el programa termina cuando se ingresa el 0
y al final da como resultado cual es el numero mayor de todos los que se ingreso y el segundo mayor
Elaborado por Maria Fernanda Paucar*/
#include <stdio.h>

int main() {
    int num, mayor, segundo_mayor, contador;
    mayor = 0;
    segundo_mayor = 0;
    contador = 0;
    printf("Ingrese un numero entero positivo (0 para terminar): ");
    scanf("%d", &num);
    while (num != 0) {
        if (num < 0) {
            printf("Por favor, ingrese solo numeros positivos.\n");
        } else {
            contador++;

            if (num > mayor) {
                segundo_mayor = mayor;
                mayor = num;
            } else if (num > segundo_mayor) {
                segundo_mayor = num;
            }
        }
        printf("Ingrese otro numero entero positivo (0 para terminar): ");
        scanf("%d", &num);
    }
    if (contador < 2) {
        printf("Error: Se necesitan al menos dos valores positivos para calcular el segundo mayor.\n"
    } else {
        printf("El numero mayor es: %d\n", mayor);
        printf("El segundo numero mayor es: %d\n", segundo_mayor);
    }

    return 0;
}
```

```
Ingrese un numero entero positivo (0 para terminar): 6
Ingrese otro numero entero positivo (0 para terminar): 5
Ingrese otro numero entero positivo (0 para terminar): 7
Ingrese otro numero entero positivo (0 para terminar): 2
Ingrese otro numero entero positivo (0 para terminar): 9
Ingrese otro numero entero positivo (0 para terminar): 0
El numero mayor es: 9
El segundo numero mayor es: 7

Process returned 0 (0x0)   execution time : 4.420 s
Press any key to continue.
```

Problema 1.20. Adivina el número

Desarrolla un programa que “piense” un número entre 1 y 100 (puede ser fijo para la práctica) y pida al usuario adivinarlo. Después de cada intento debe indicar “más alto”, “más bajo” o “¡acertaste!”, y al final mostrar cuántos intentos se usaron.
Pista: Usa un ciclo que se repita hasta que el intento sea igual al número secreto, y un contador de intentos.

Análisis del problema

- Entrada: El número que escribe el usuario para intentar adivinar (intento).
- Proceso: Guardamos el número que hay que adivinar en una variable (secreto = 42). Ponemos un contador de intentos en cero (intentos = 0).Pedimos el primer número. Usamos un ciclo Mientras que se va a repetir mientras el usuario no adivine el número (es decir, mientras intento != secreto). Dentro del ciclo, si el número es chico le decimos "más alto", y si es grande le decimos "más bajo". Luego sumamos 1 a los intentos y volvemos a pedir otro número.
- Salida: Cuando por fin adivina y sale del ciclo, le decimos "¡Acertaste!" y cuántos intentos usó.

Tabla de objetos

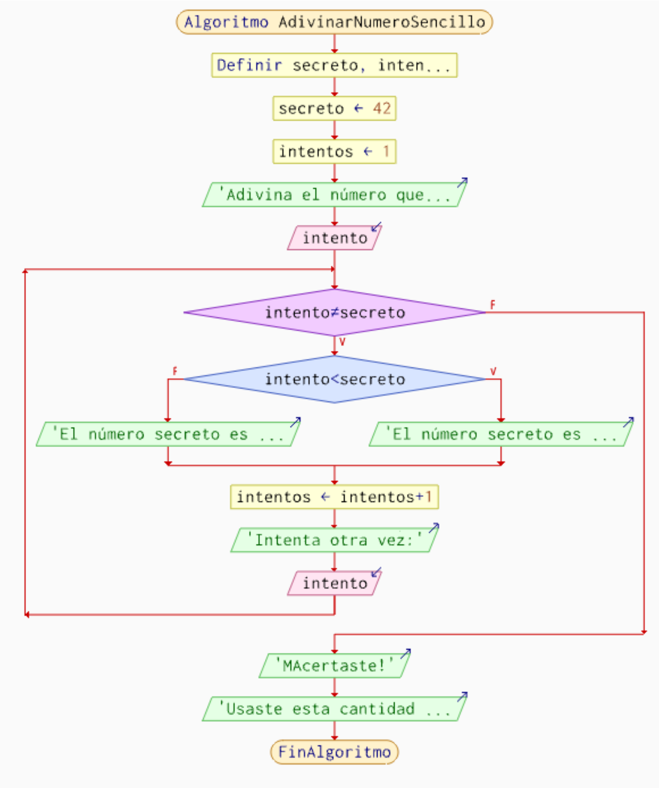
OBJETO	NOMBRE	VALOR	TIPO
El numero a adivinar	Secreto	Vale 42 (numero fijo)	Entero
Numero que escribe el usuario	Intento	Lo elige el usuario	Entero
Contador de intentos	Intentos	Empieza en 1	Entero

Seudocódigo

```
Algoritmo AdivinarNumeroSencillo

  Definir secreto, intento, intentos Como Entero

  secreto <- 42

  intentos <- 1

  Escribir "Adivina el número que pensé entre 1 y 100:"

  Leer intento

  Mientras intento <> secreto Hacer


    Si intento < secreto Entonces

      Escribir "El número secreto es más alto."

    Sino

      Escribir "El número secreto es más bajo."

    FinSi

    intentos <- intentos + 1
```

```
        Escribir "Intenta otra vez:"

        Leer intento


    FinMientras

    Escribir "¡Acertaste!"

    Escribir "Usaste esta cantidad de intentos: ", intentos

FinAlgoritmo
```

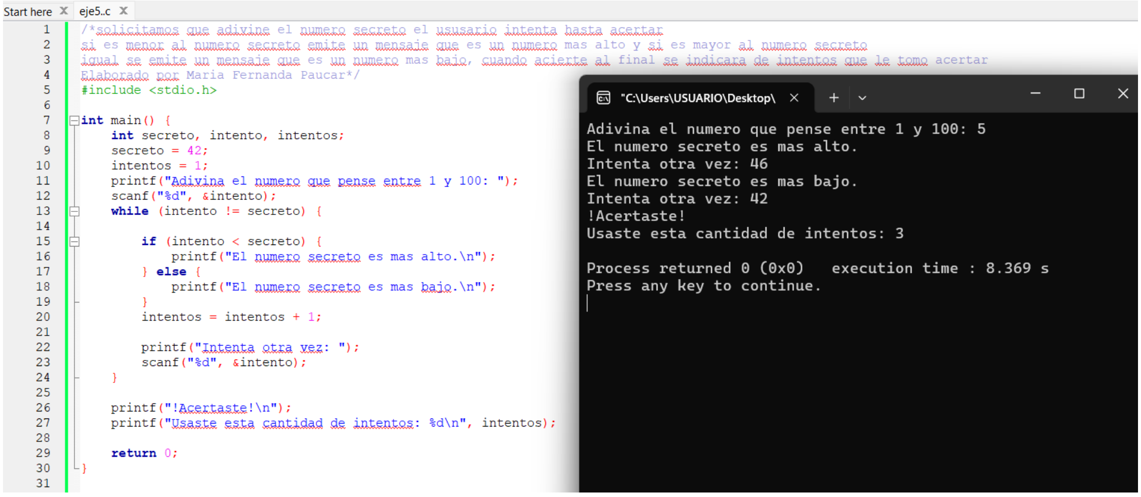
Diagrama de flujo


Algoritmo AdivinarNumeroSencillo
Definir secreto, inten...
secreto ← 42
intentos ← 1
'Adivina el número que...
intento
intento≠secreto
F
V
intento<secreto
F
V
'El número secreto es ...
'El número secreto es ...
intentos ← intentos+1
'Intenta otra vez:'
intento
'MAcertaste!'
'Usaste esta cantidad ...
FinAlgoritmo

Prueba de escritorio

Jugada	Intento	Intento!= secreto	Pista en pantalla	Intentos
1	20	Sí (20 ≠ 42)	El numero secreto es mas alto	1
2	50	Sí (50 ≠ 42)	El numero secreto es mas bajo	2
3	42	No (42 ≠ 42 es Falso)	No entra en el bucle	3

Salida: ACERTASTE usaste esta cantidad de intentos:3



```c
/*solicitamos que adivine el numero secreto el ususario intenta hasta acertar
si es menor al numero secreto emite un mensaje que es un numero mas alto y si es mayor al numero secreto
igual se emite un mensaje que es un numero mas bajo, cuando acierte al final se indicara de intentos que le tomo acertar
Elaborado por Maria Fernanda Paucar*/
#include <stdio.h>

int main() {
    int secreto, intento, intentos;
    secreto = 42;
    intentos = 1;
    printf("Adivina el numero que pense entre 1 y 100: ");
    scanf("%d", &intento);
    while (intento != secreto) {

        if (intento < secreto) {
            printf("El numero secreto es mas alto.\n");
        } else {
            printf("El numero secreto es mas bajo.\n");
        }
        intentos = intentos + 1;

        printf("Intenta otra vez: ");
        scanf("%d", &intento);
    }

    printf("!Acertaste!\n");
    printf("Usaste esta cantidad de intentos: %d\n", intentos);

    return 0;
}

```

```
Adivina el numero que pense entre 1 y 100: 5
El numero secreto es mas alto.
Intenta otra vez: 46
El numero secreto es mas bajo.
Intenta otra vez: 42
!Acertaste!
Usaste esta cantidad de intentos: 3

Process returned 0 (0x0)   execution time : 8.369 s
Press any key to continue.
```